

POLITECNICO MILANO 1863 | DIPARTIMENTO DI ELETTRONICA INFORMAZIONE E BIOINGEGNERIA

Automation and Control in Autonomous Vehicles (ACAV)

Visual Perception for Autonomous Vehicles

AY 2026-2027

Prof. Matteo Corno
move.deib.polimi.it

Outline

Visual Perception for Autonomous Vehicles

1. Introduction to Cameras
2. Camera Projective Geometry
3. Model Based Image Processing
4. Neural Network Image Processing
5. Conclusions

Outline

Visual Perception for Autonomous Vehicles

1. Introduction to Cameras
2. **Camera Projective Geometry**
3. Model Based Image Processing
4. Neural Network Image Processing
5. Conclusions

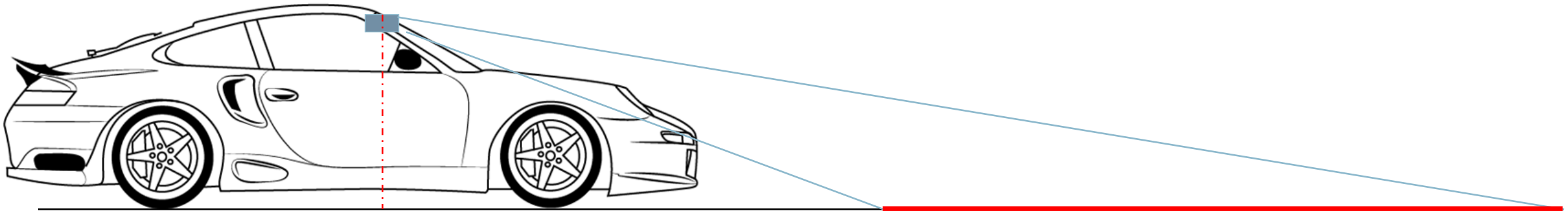
Bird-Eye View

$$sO_{image} = \begin{bmatrix} f & 0 & c_x & 0 \\ 0 & f & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} O_{world}$$

In general, this is not invertible.

Bird-Eye View

In general, this is not invertible.



Bird-Eye View



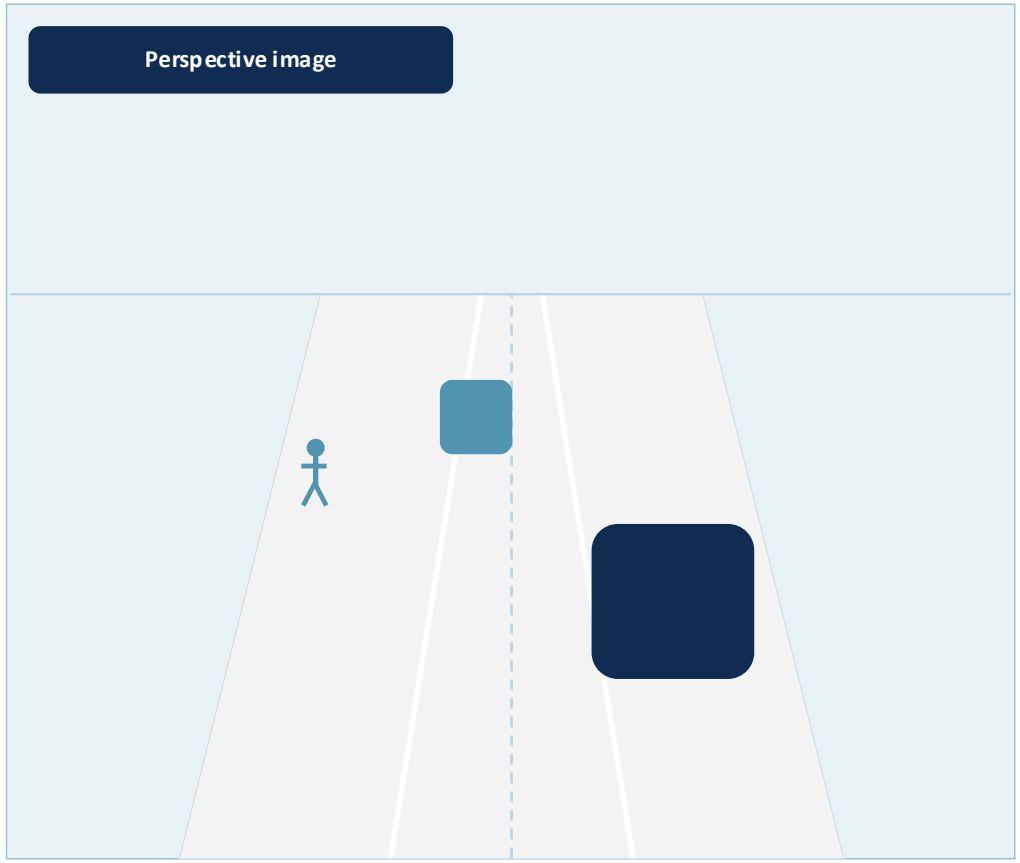




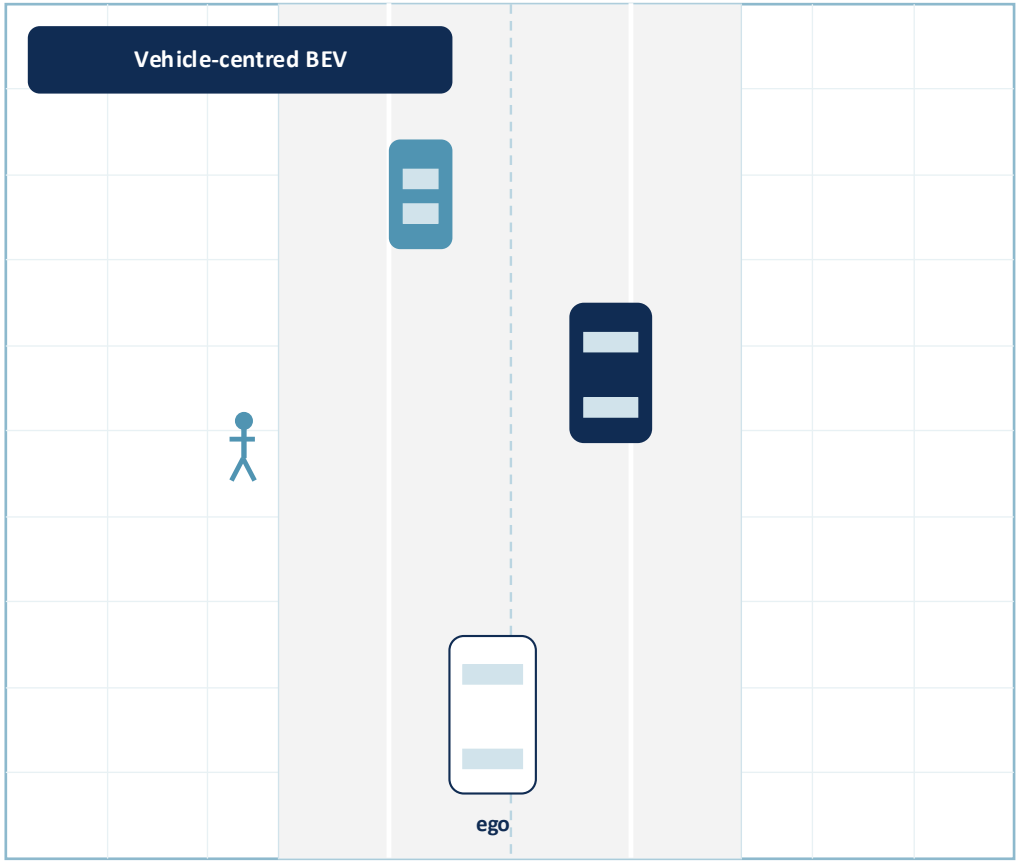




Bird-Eye View



Appearance changes with depth

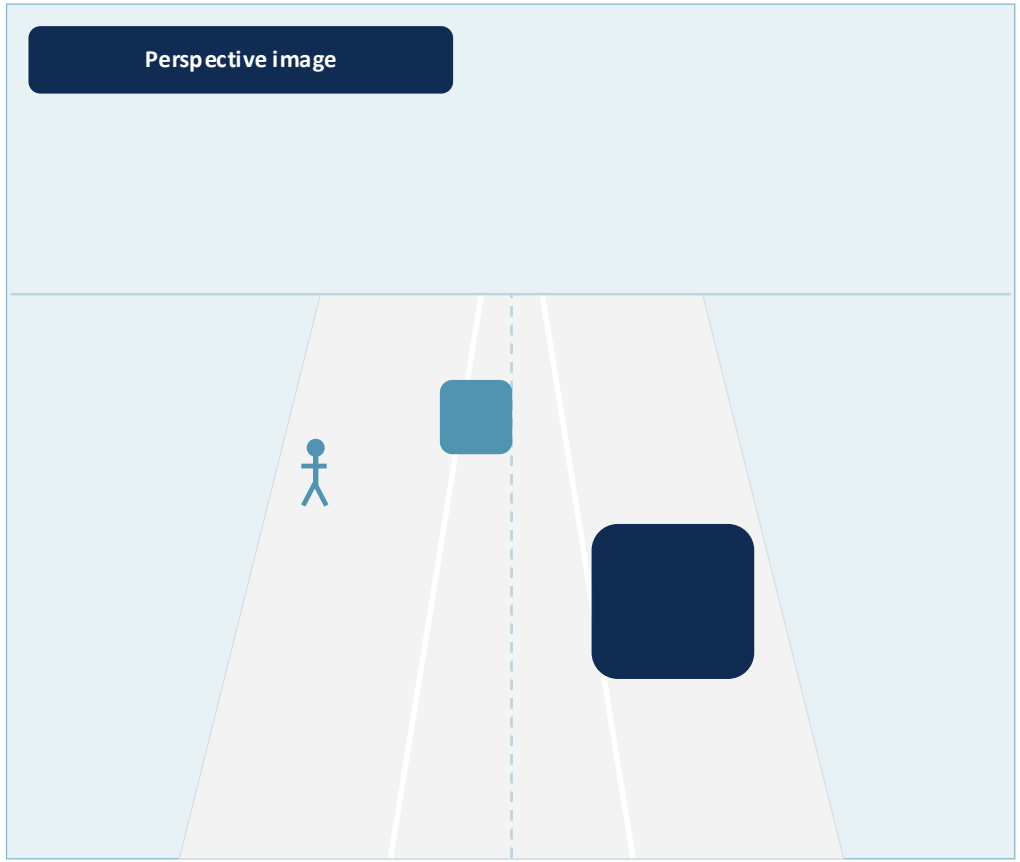


Distances and directions become explicit

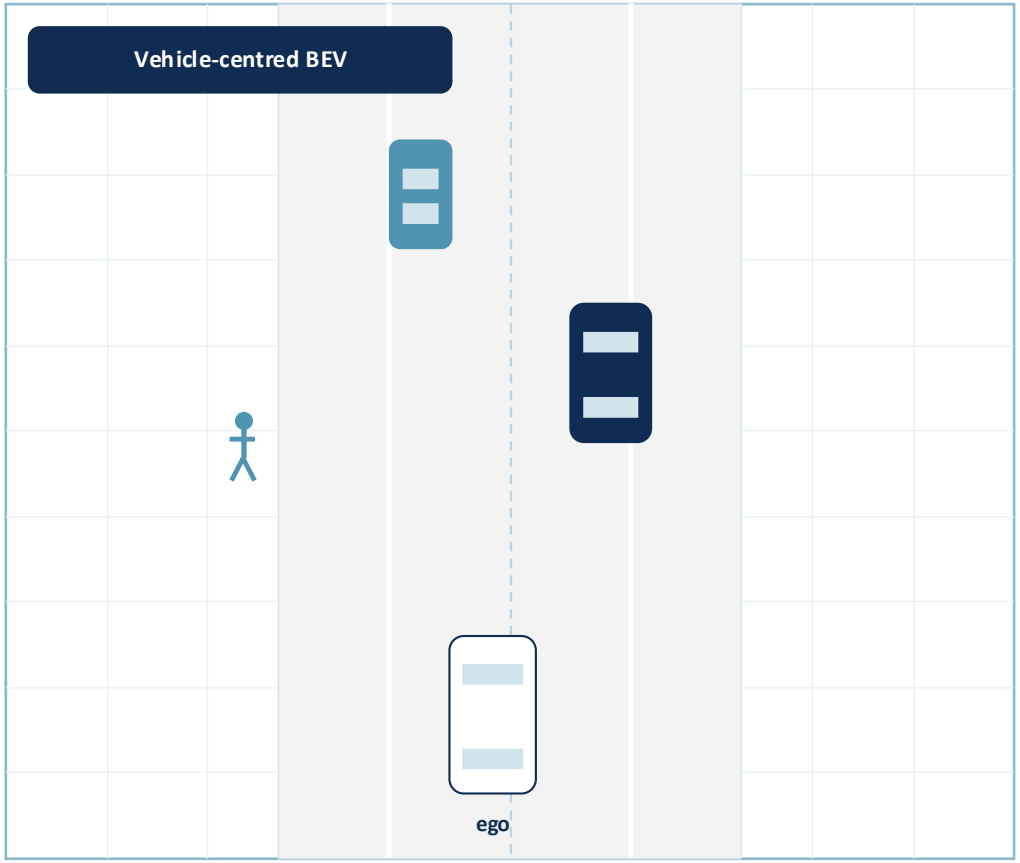
Two approaches:

- Model based
- ML/AI based

Bird-Eye View



Appearance changes with depth



Distances and directions become explicit

Two approaches:

- **Model based**
- ML/AI based

Bird-Eye View

$$sO_{image} = \begin{bmatrix} f & 0 & c_x & 0 \\ 0 & f & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} O_{world}$$

In general, this is not invertible.

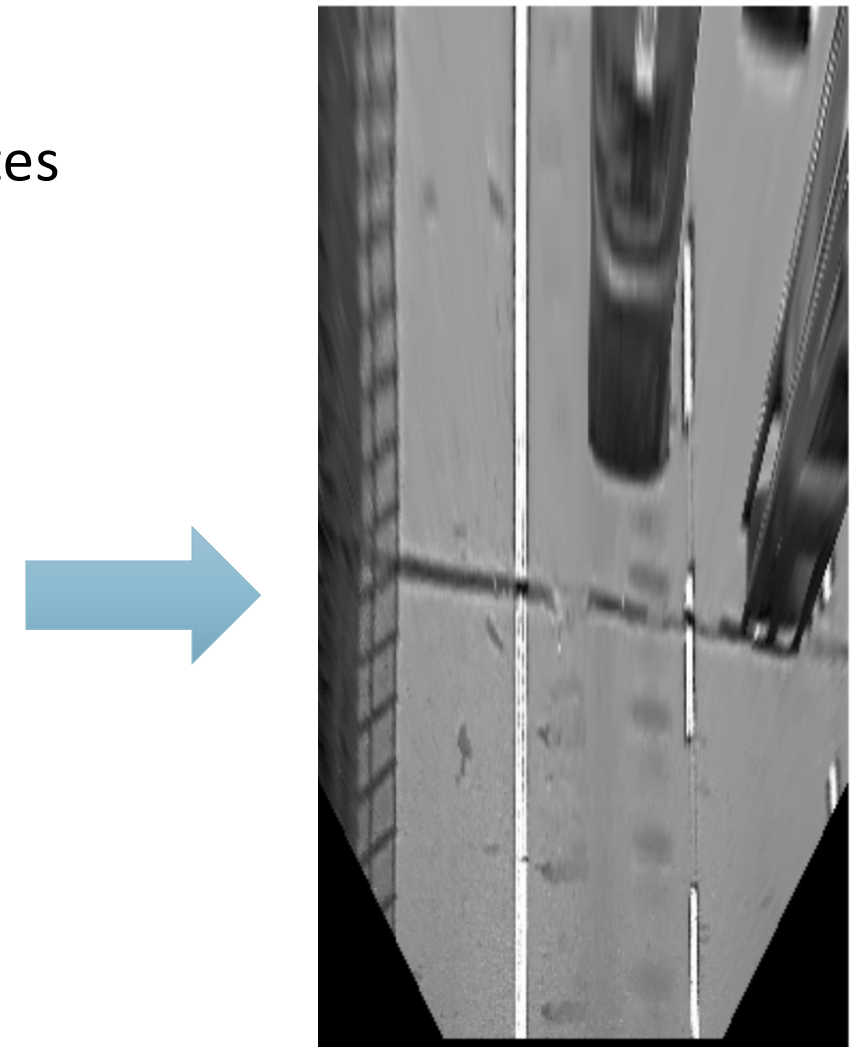
I assume that the world coordinates all have $z_w = 0$, it becomes a projection from R^2 to R^2 and it is invertible.

→ Inverse Perspective Mapping

Bird-Eye View

In general, this is not invertible.
I assume that the world coordinates all have

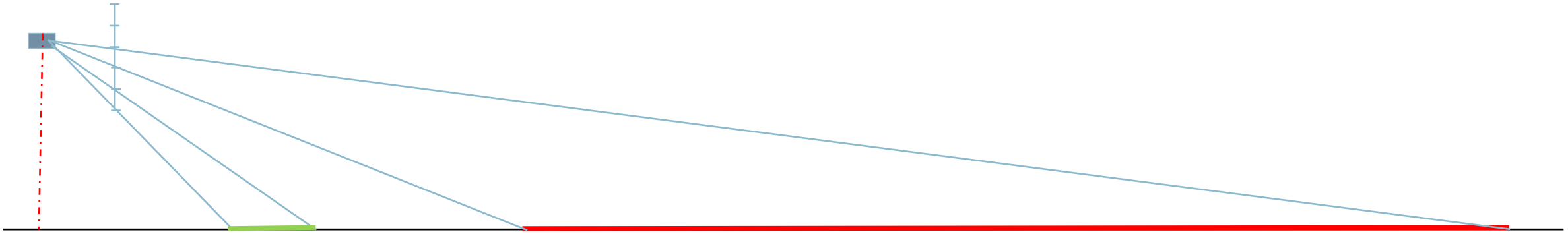




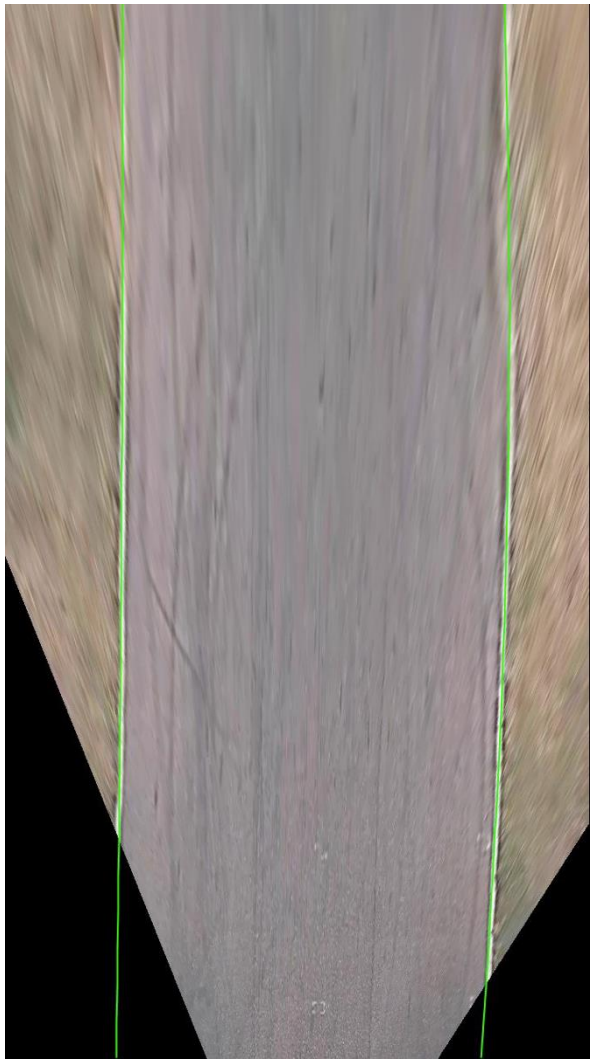
- Other cars are distorted because not at the right z
- The areas further away from the sensor are blurrier, due to having fewer pixels
- The width of the lanes is constant (no perspective)

Bird-Eye View

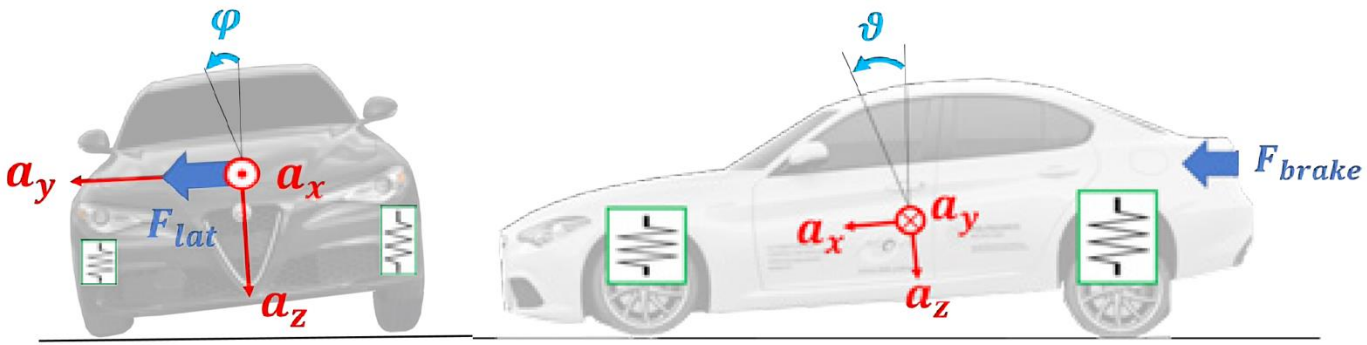
- The areas further away from the sensor are blurrier, due to having fewer pixels



Bird-Eye View



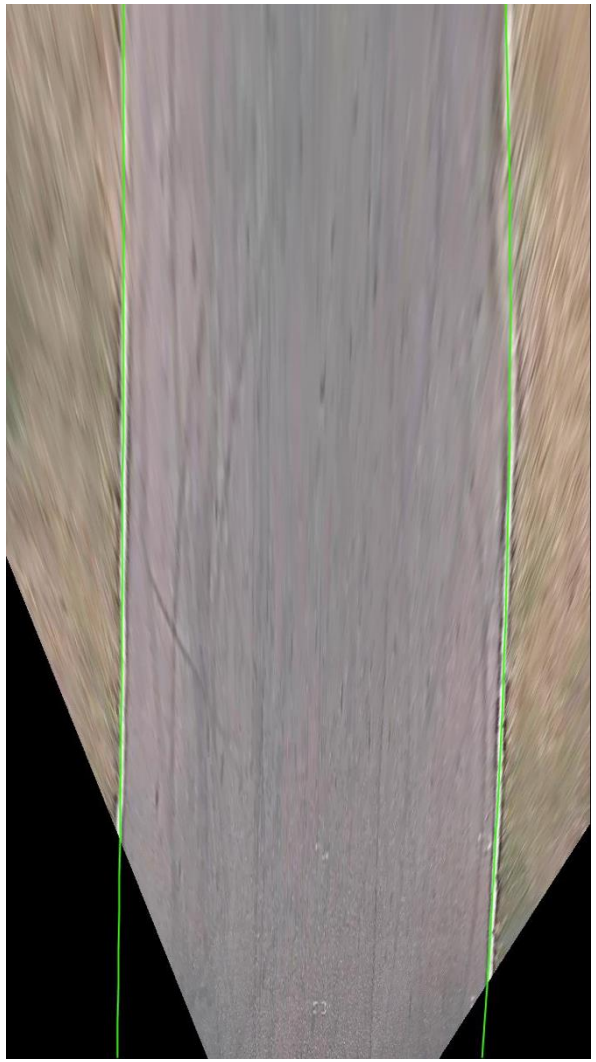
Bird-Eye View



(a) Roll angle (b) Pitch angle

$$\varphi \approx -\frac{Mh}{K^s L_\varphi^2} a_y = \gamma_\varphi a_y$$

$$\theta \approx -\frac{Mh}{K^s L_\theta^2} a_x = \gamma_\theta a_x$$



Bird-Eye View

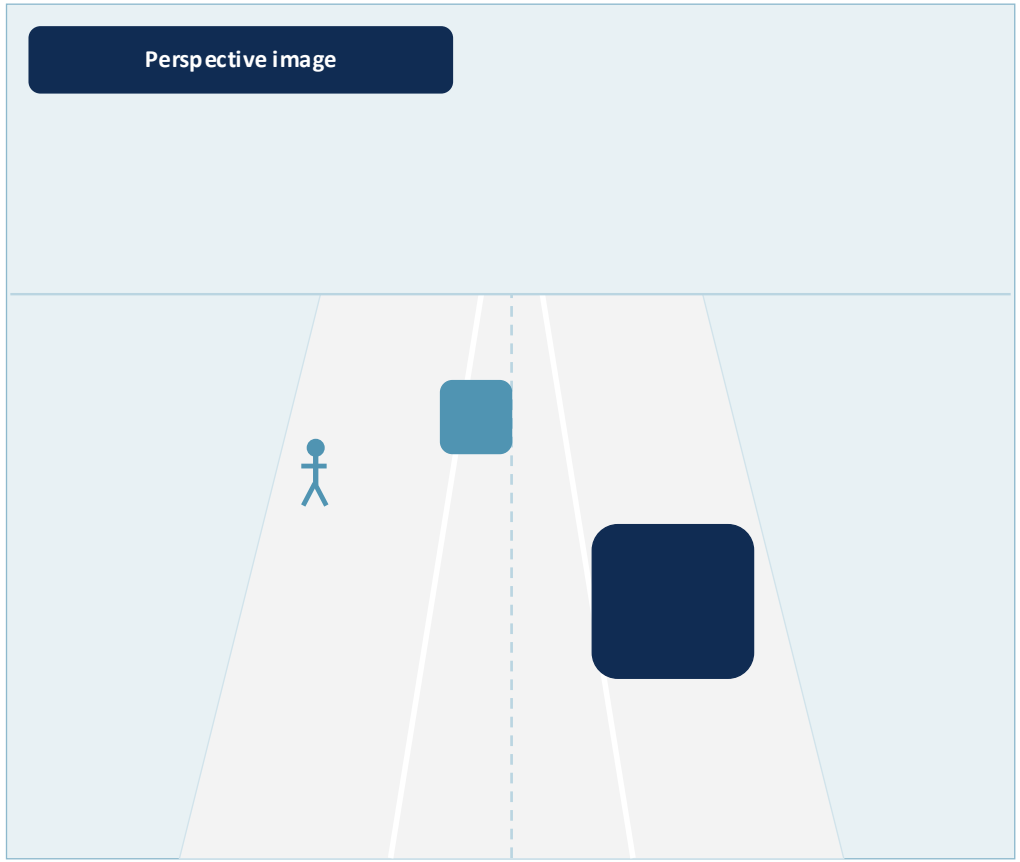


Before vehicle dynamics correction

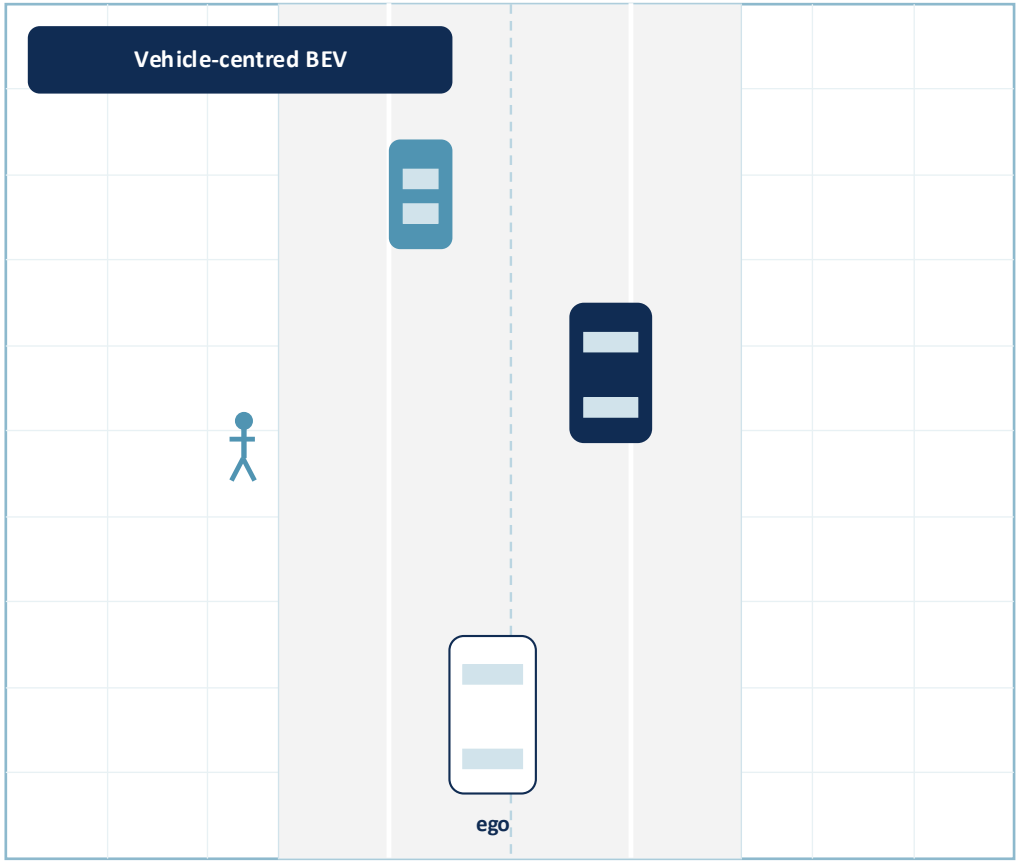


After vehicle dynamics correction

Bird-Eye View



Appearance changes with depth

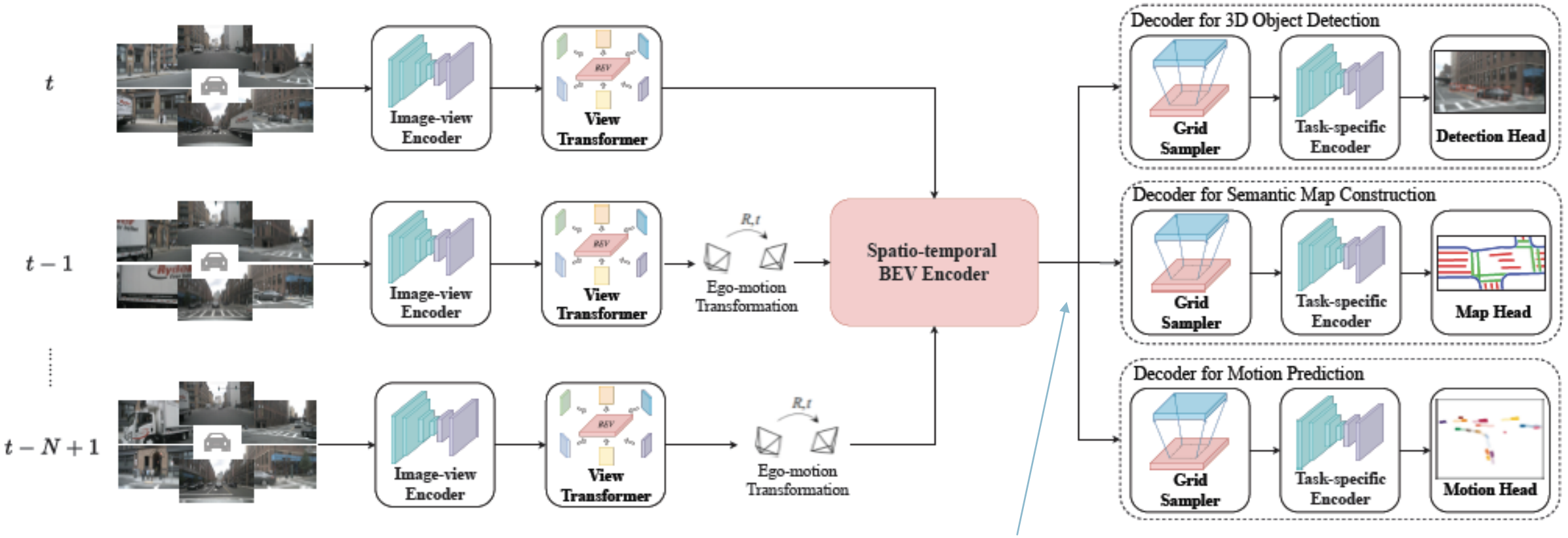


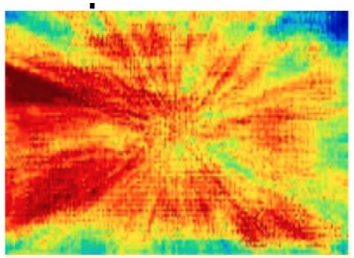
Distances and directions become explicit

Two approaches:

- Model based
- **ML/AI based**

Bird-Eye View

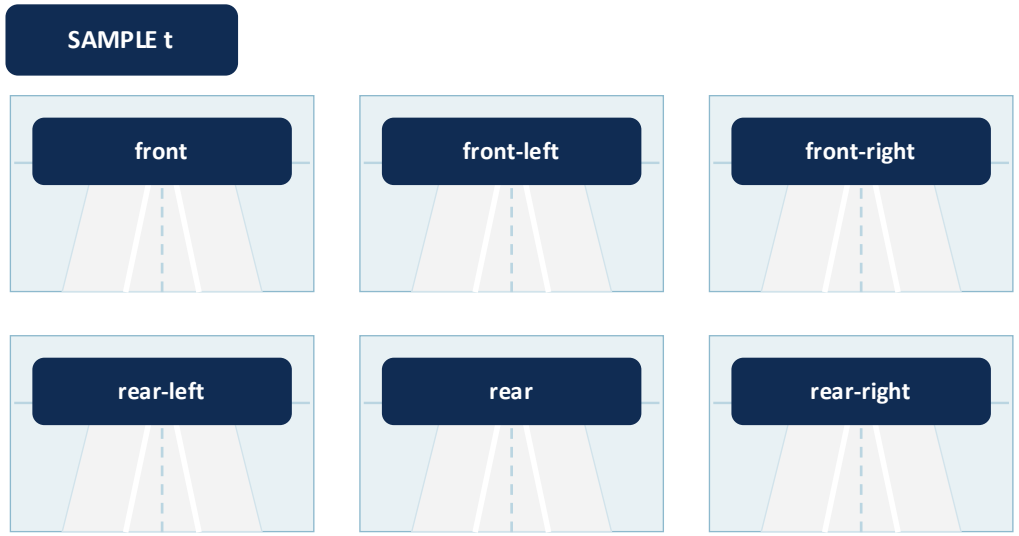




Camera BEV Features

Latent space (features)

Bird-Eye View



K_i, $T_i \rightarrow$ego

timestamp

ego pose

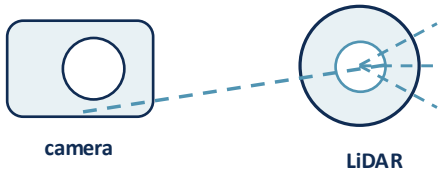
3D boxes

segmentation

All components must refer to the same physical instant and coordinate convention.

Bird-Eye View

Spatial consistency



Check that projected LiDAR points land on the observed surfaces.

Temporal consistency



At 15 m/s, 40 ms = 0.6 m of ego translation before object motion is considered.

A perfectly labeled box is still wrong if it is expressed in the wrong frame or time.

Bird-Eye View

Dataset	Scale / rig	Useful labels	BEV use
KITTI	7,481 train + 7,518 test object images; front stereo + LiDAR	2D/3D boxes, calibration	classical / front-view prototyping
nuScenes	1,000 × 20 s; 6 cameras, LiDAR, 5 radars	3D boxes, tracks, maps, lidarseg	canonical surround BEV
Waymo Open	1,150 × 20 s; 5 cameras, 5 LiDARs	2D/3D boxes, tracks, segmentation, maps	scale and geographic diversity
Argoverse 2	1,000 × 15 s; 7 ring + 2 stereo cameras, LiDAR	3D cuboids, ground height, HD maps	mapping and cross-city study
Occ3D	derived from nuScenes / Waymo	dense semantic occupancy + visibility	3D occupancy supervision

Classical and learned BEV: side-by-side

Criterion	Classical BEV	Learned BEV
Core mechanism	calibration + geometry + rules	geometry-aware neural feature transformation
Training data	little or none	large synchronized and labeled corpus
Geometry	exact under assumptions	learned uncertainty beyond assumptions
Semantics	external detector / handcrafted logic	jointly learned from task supervision
Compute	low and predictable	moderate to high; hardware dependent
Explainability	direct failure model	requires probes, slices and uncertainty
Generalization	good when assumptions stay valid	stronger scope, but vulnerable to domain shift
Failure behavior	often abrupt at assumption violation	can be graceful or confidently wrong

Outline

Visual Perception for Autonomous Vehicles

1. Introduction to Cameras
2. Camera Projective Geometry
3. **Model Based Image Processing**
4. Neural Network Image Processing
5. Conclusions

Model Based Image Processing

In model (of feature) Based Image Processing, we leverage apriori knowledge on the scenario.

Examples:

- QR code
- Lane detection
- Road detection
- Pole detection









These algorithms look for specific features

Model Based Image Processing

A lane is *"a division of a road marked off with painted lines and intended to separate single lines of traffic according to speed or direction."*

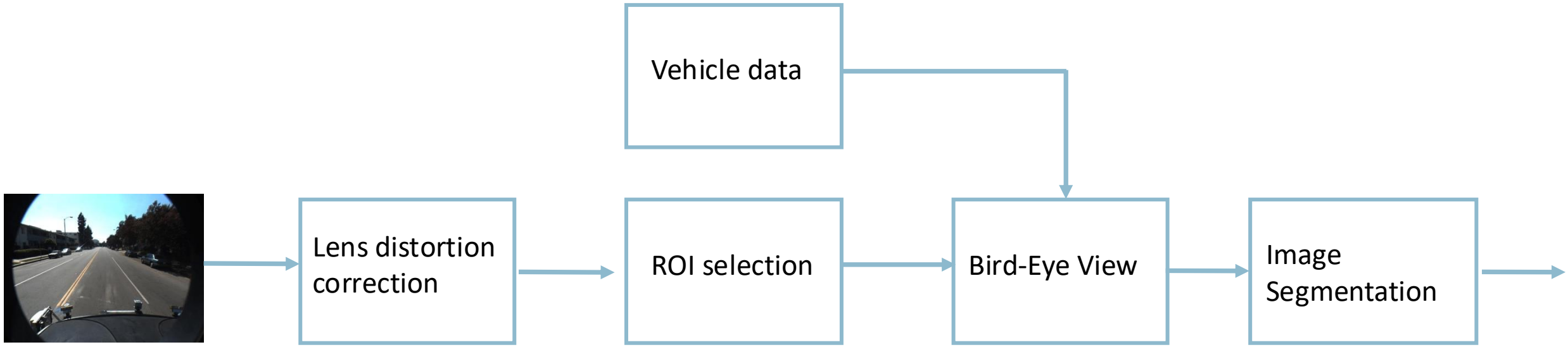
Painted lines have two main characteristics:

- Their color is different from that of the road
- They appear straight (or almost straight)

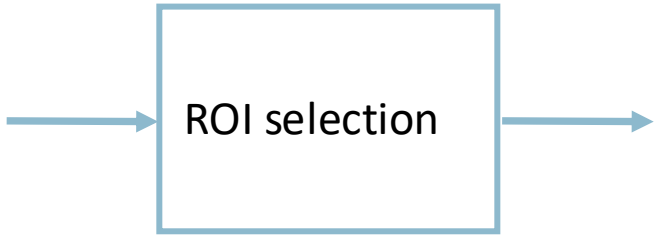


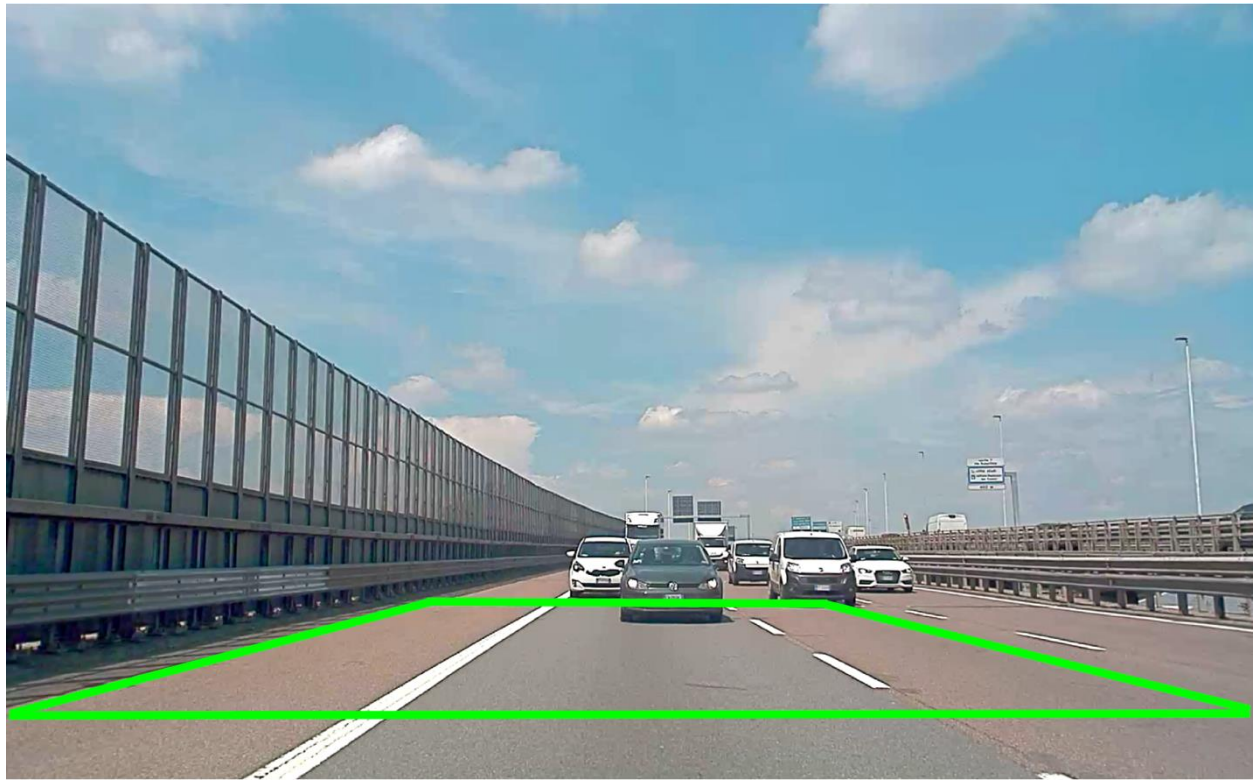
An algorithm can exploit these two features

Model Based Image Processing



Model Based Image Processing





We use some qualitative information on the properties of the camera:

- Field of view
- Position and orientation

Image Segmentation



The image segmentation is based on the fact that the line color is different from that of the road.

I could work with the color intensity (drawbacks?)

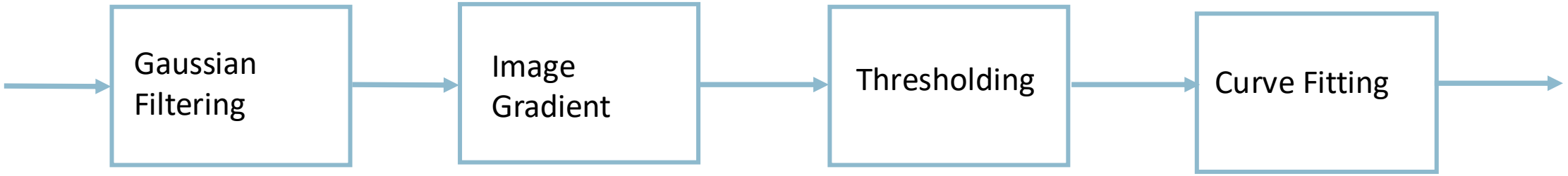
Lane Segmentation



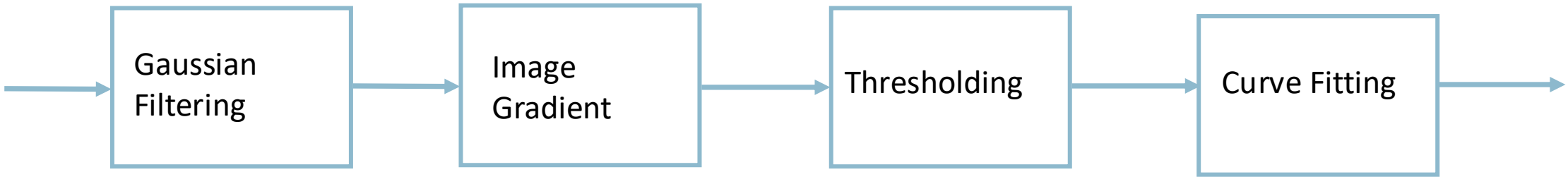
The image segmentation is based on the fact that the line color is different from that of the road.

I could work with the color intensity (drawbacks?)
or
I could consider the discontinuities → High pass filtering and then threshold.

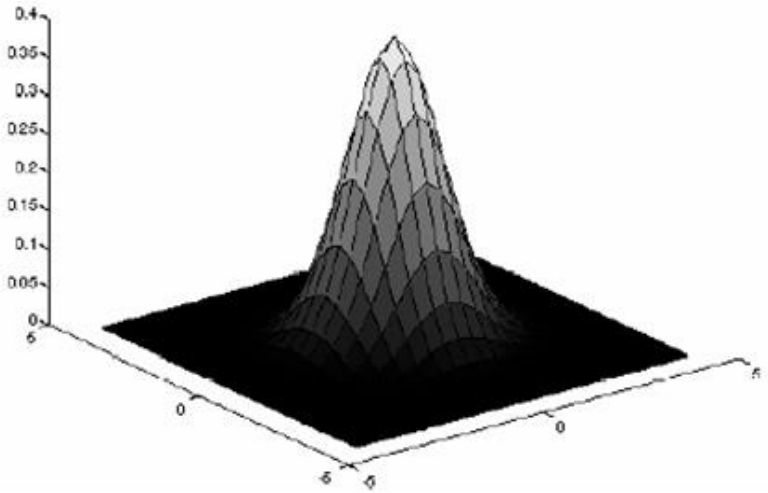
The image is a function of in R^2

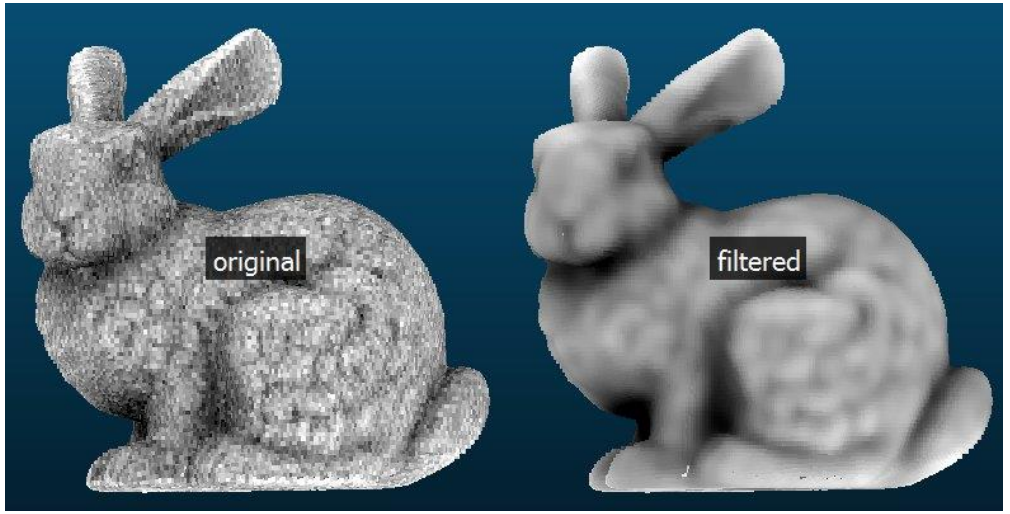


Lane Segmentation

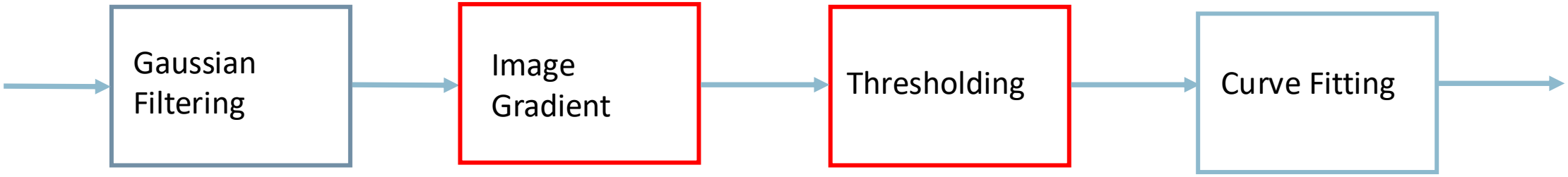


Filtering is performed by convoluting the image with a Gaussian template → each pixel of the new image is an average of the neighboring pixels weighted with the Gaussian shape



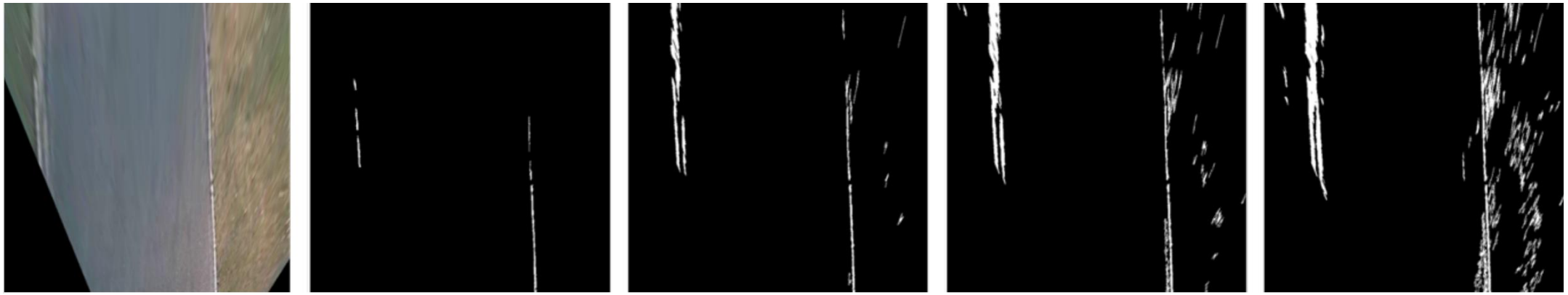


Lane Segmentation

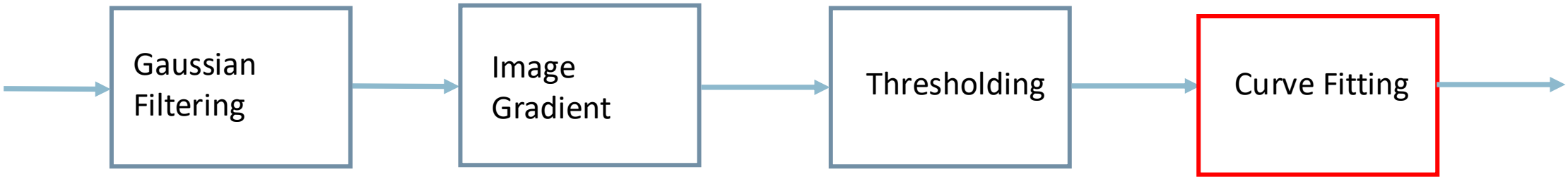


Gradient computation is performed by convoluting the image with two templates

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad G = \sqrt{G_x^2 + G_y^2} \quad I_2 = G > \bar{I}$$

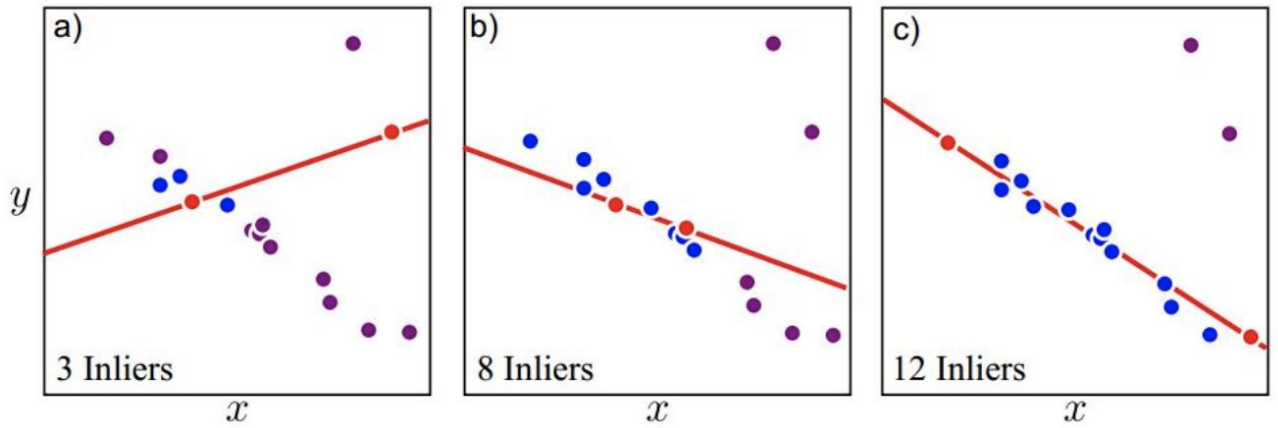


Lane Segmentation

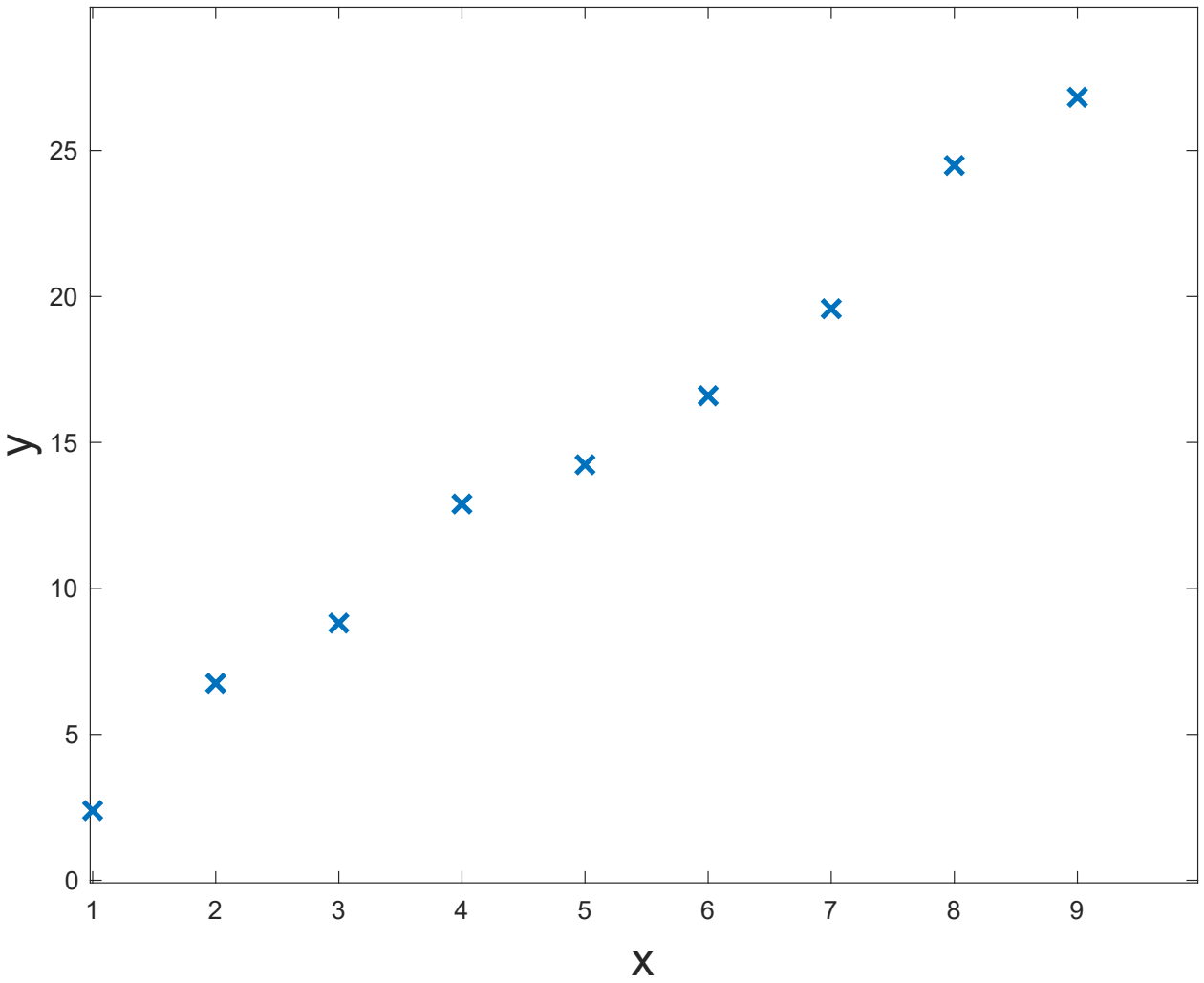


RAndom Sample Consensus (RANSAC)

- Can be used with different models
- Computationally efficient
- RANSAC is non deterministic.
- It solves the fitting problem many times using randomly selected minimal subsets of the data and choosing the best fit.

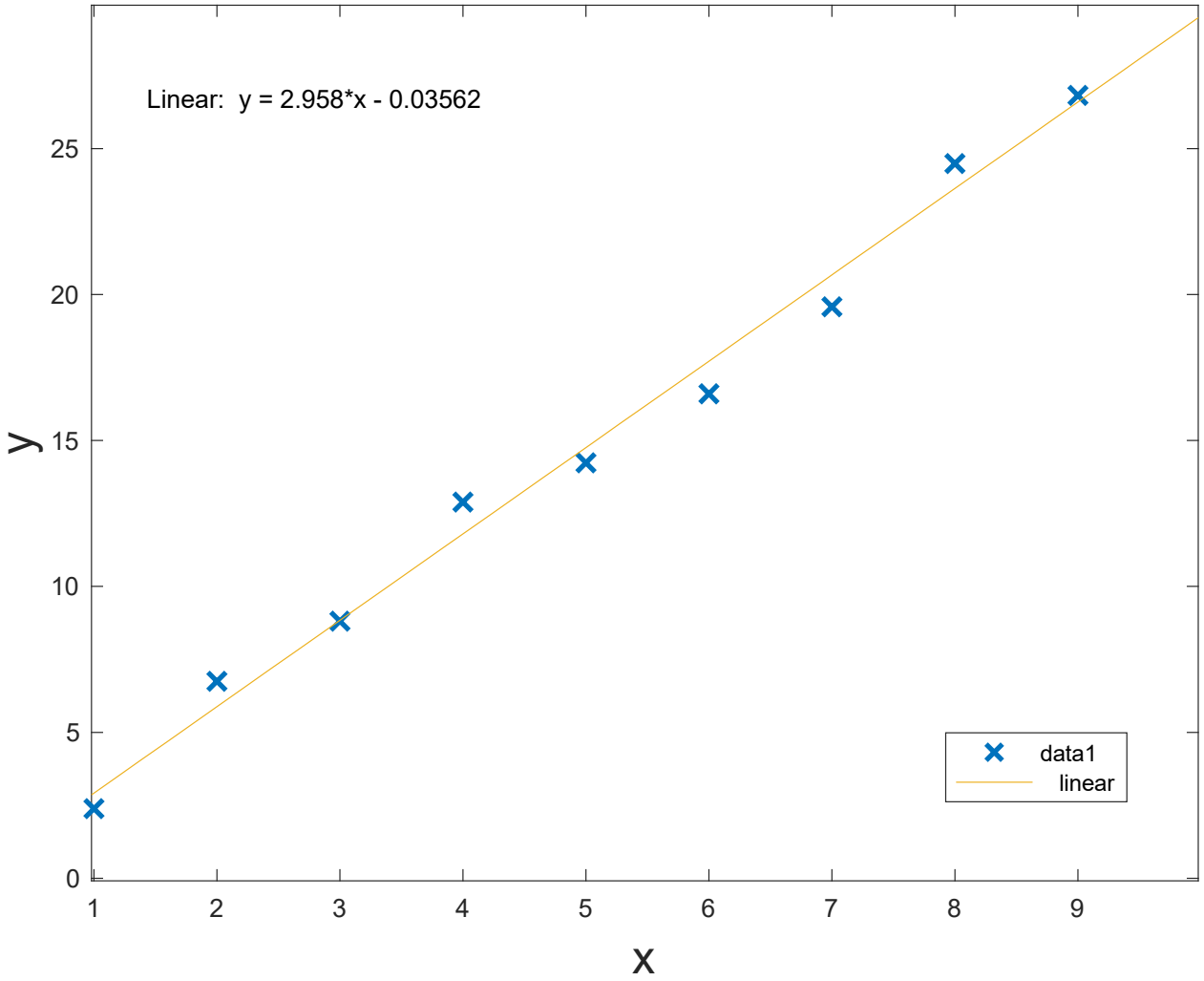


Curve Fitting



$$y = 3x + v$$

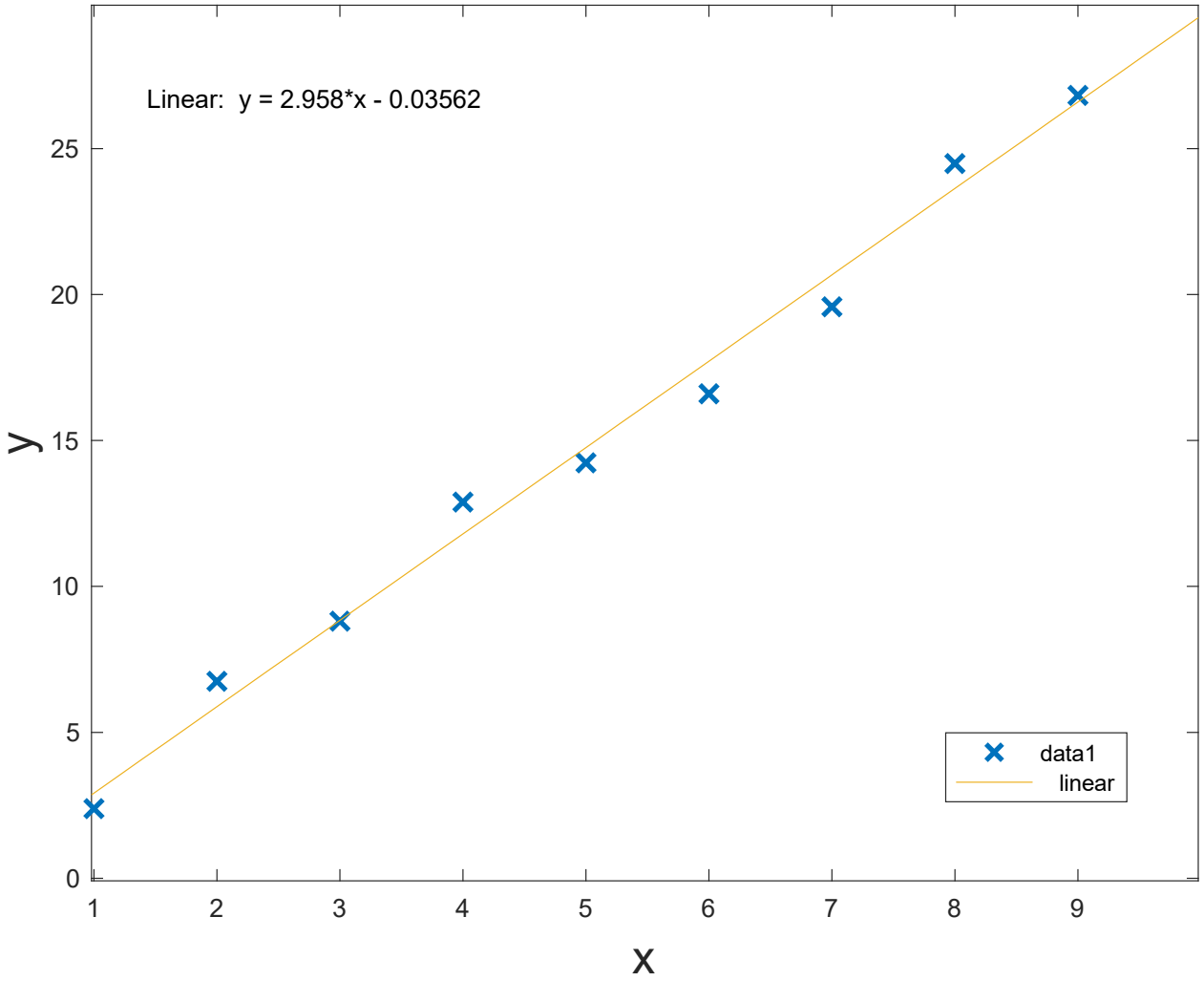
Curve Fitting



$$y = 3x + v$$

$$\min_{a,b} \sum_{i=1}^{N} (y_{data} - a x_{data} - b)^2$$

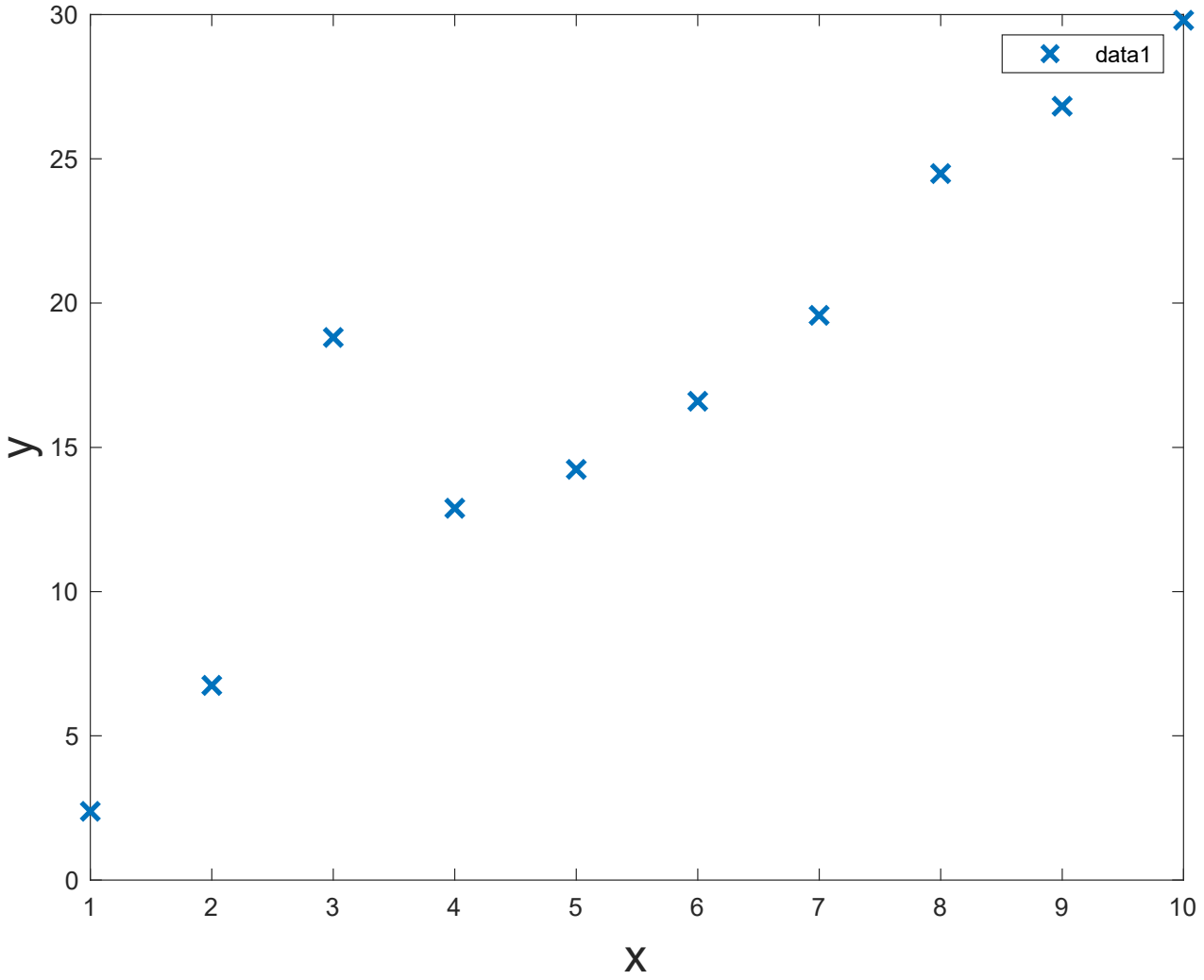
Curve Fitting



$$y = 3x + v$$

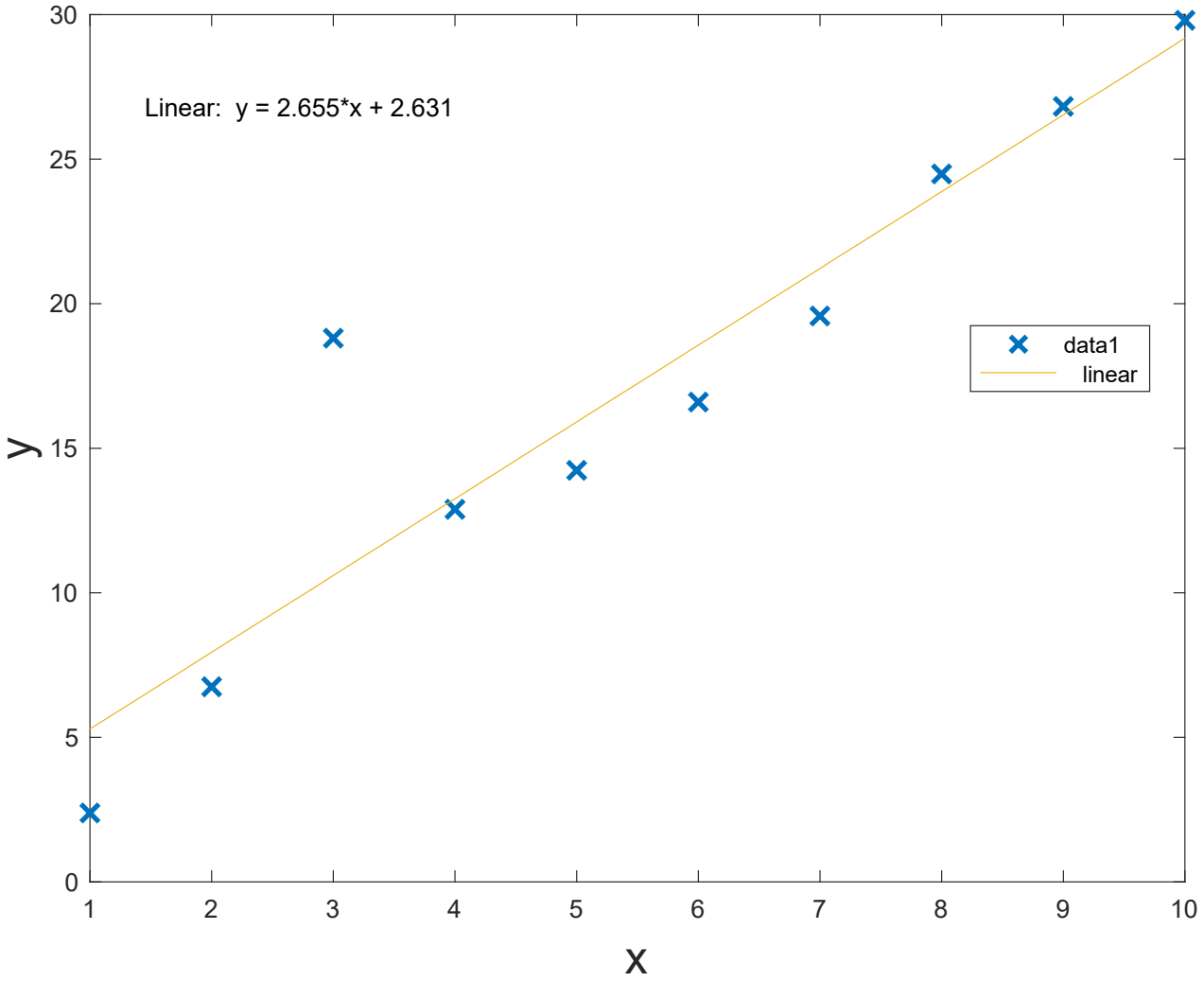
$$\min_{a,b} \sum_{i=1}^{N} (y_{data} - a x_{data} - b)^2$$

Curve Fitting



$$y = 3x + v$$

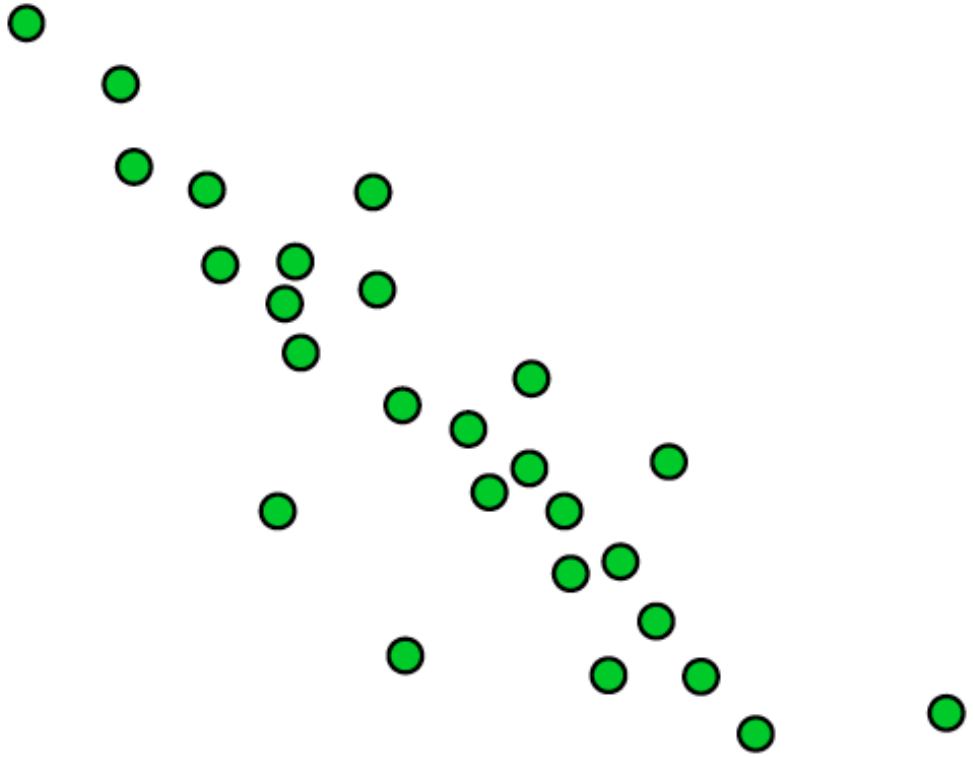
Curve Fitting



$$y = 3x + v$$

Classical Least Square is not robuts to ouliers

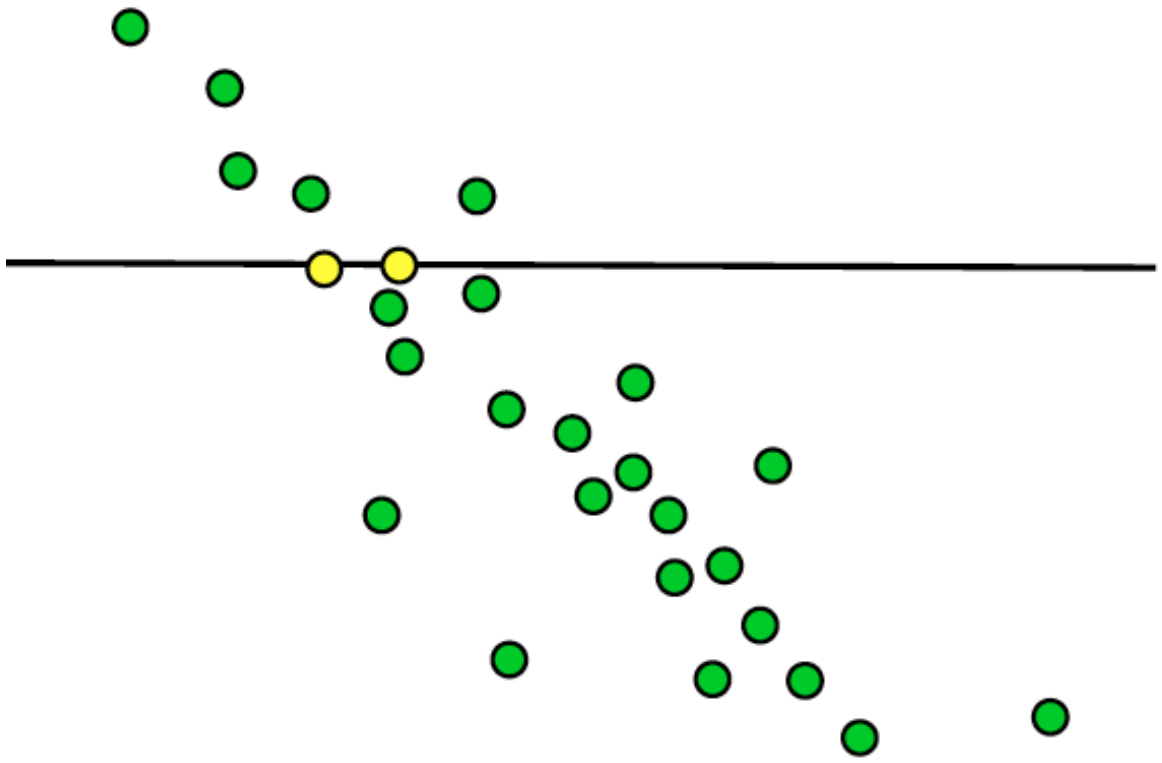
RANSAC



RAndom Sample Consensus (RANSAC)

- Can be used with different models
- Computationally efficient
- RANSAC is non deterministic.
- It solves the fitting problem many times using randomly selected minimal subsets of the data and choosing the best fit.

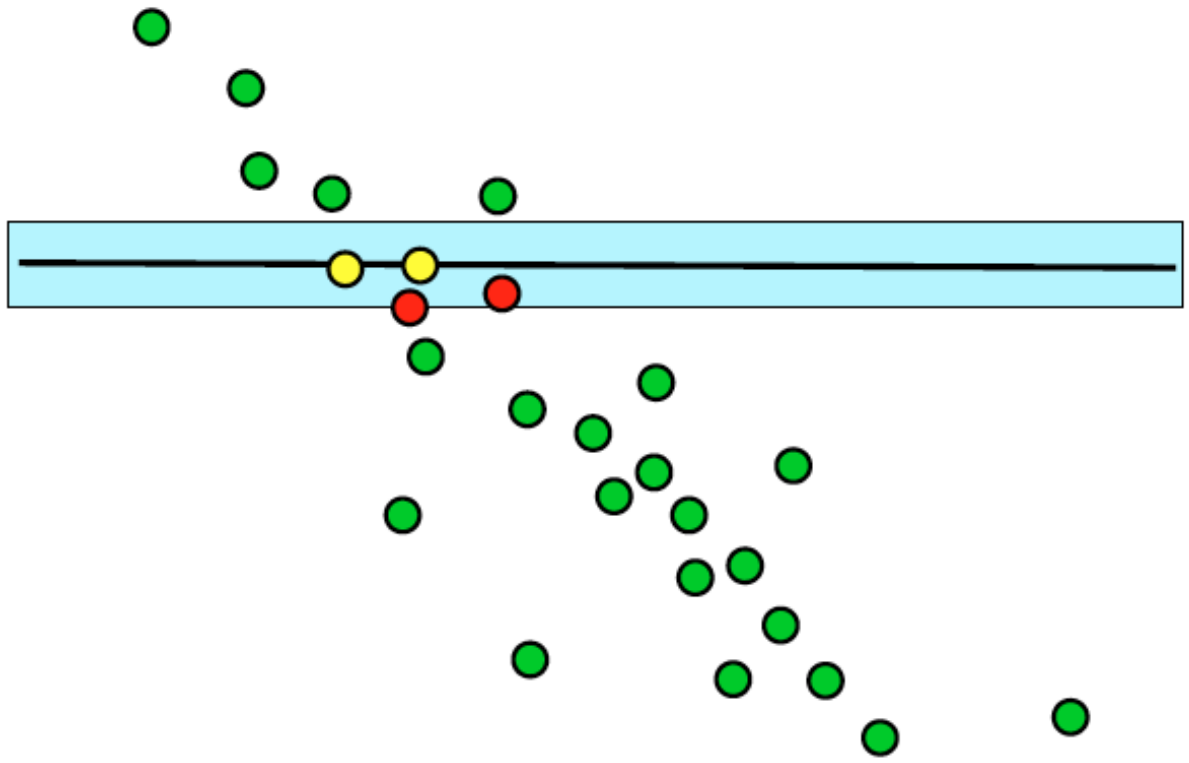
RANSAC



RAndom Sample Consensus (RANSAC)

- Can be used with different models
- Computationally efficient
- RANSAC is non deterministic.
- It solves the fitting problem many times using randomly selected minimal subsets of the data and choosing the best fit.

RANSAC

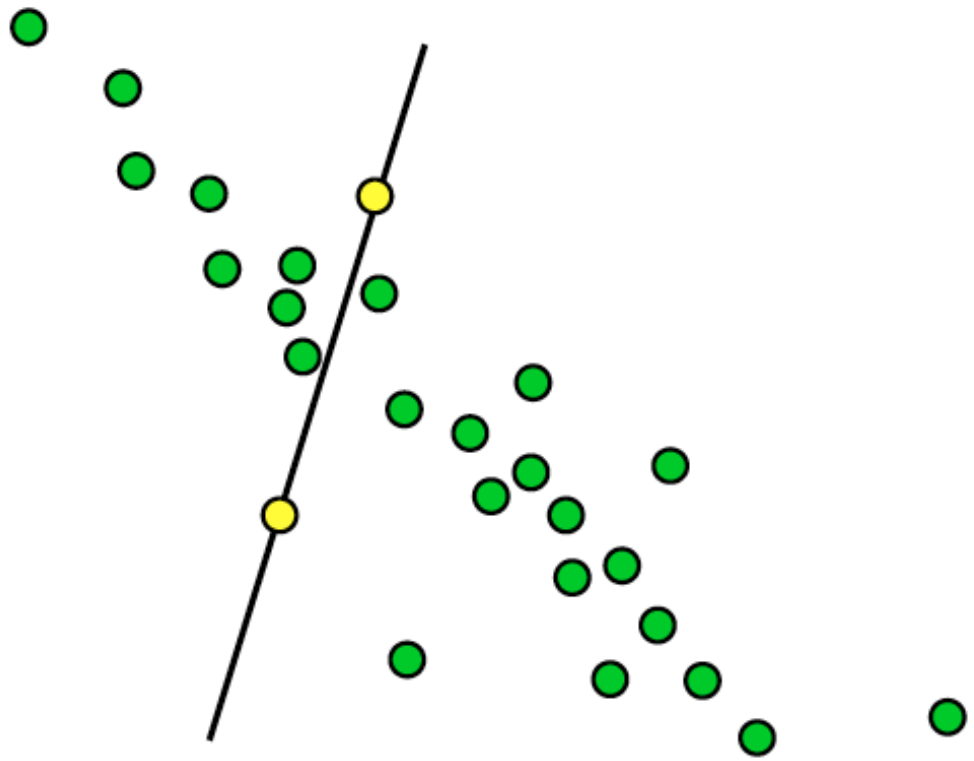


RAndom Sample Consensus (RANSAC)

- Can be used with different models
- Computationally efficient
- RANSAC is non deterministic.
- It solves the fitting problem many times using randomly selected minimal subsets of the data and choosing the best fit.

Count = 4

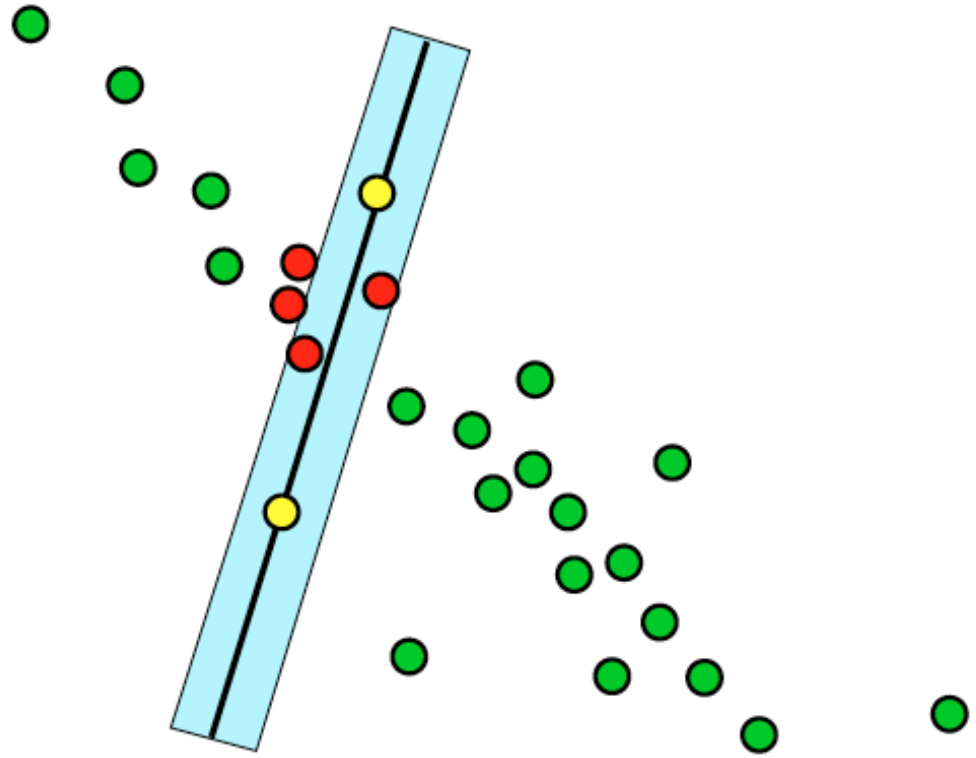
RANSAC



RAndom Sample Consensus (RANSAC)

- Can be used with different models
- Computationally efficient
- RANSAC is non deterministic.
- It solves the fitting problem many times using randomly selected minimal subsets of the data and choosing the best fit.

RANSAC

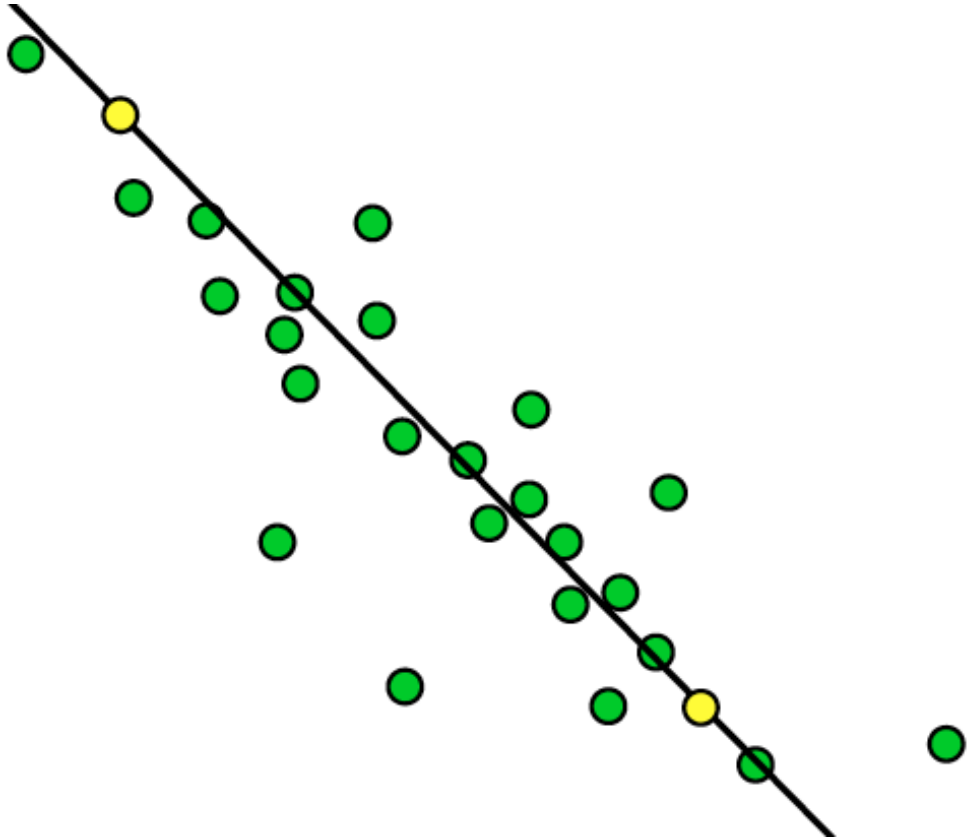


RAndom Sample Consensus (RANSAC)

- Can be used with different models
- Computationally efficient
- RANSAC is non deterministic.
- It solves the fitting problem many times using randomly selected minimal subsets of the data and choosing the best fit.

Count = 6

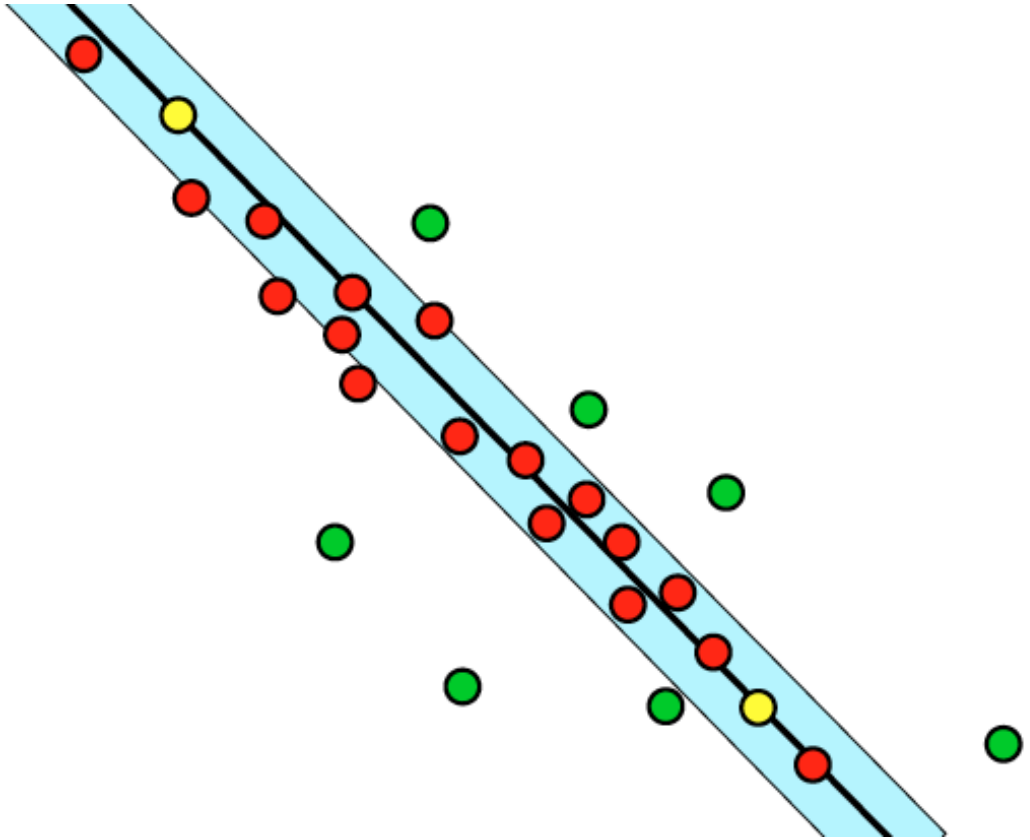
RANSAC



RAndom Sample Consensus (RANSAC)

- Can be used with different models
- Computationally efficient
- RANSAC is non deterministic.
- It solves the fitting problem many times using randomly selected minimal subsets of the data and choosing the best fit.

RANSAC

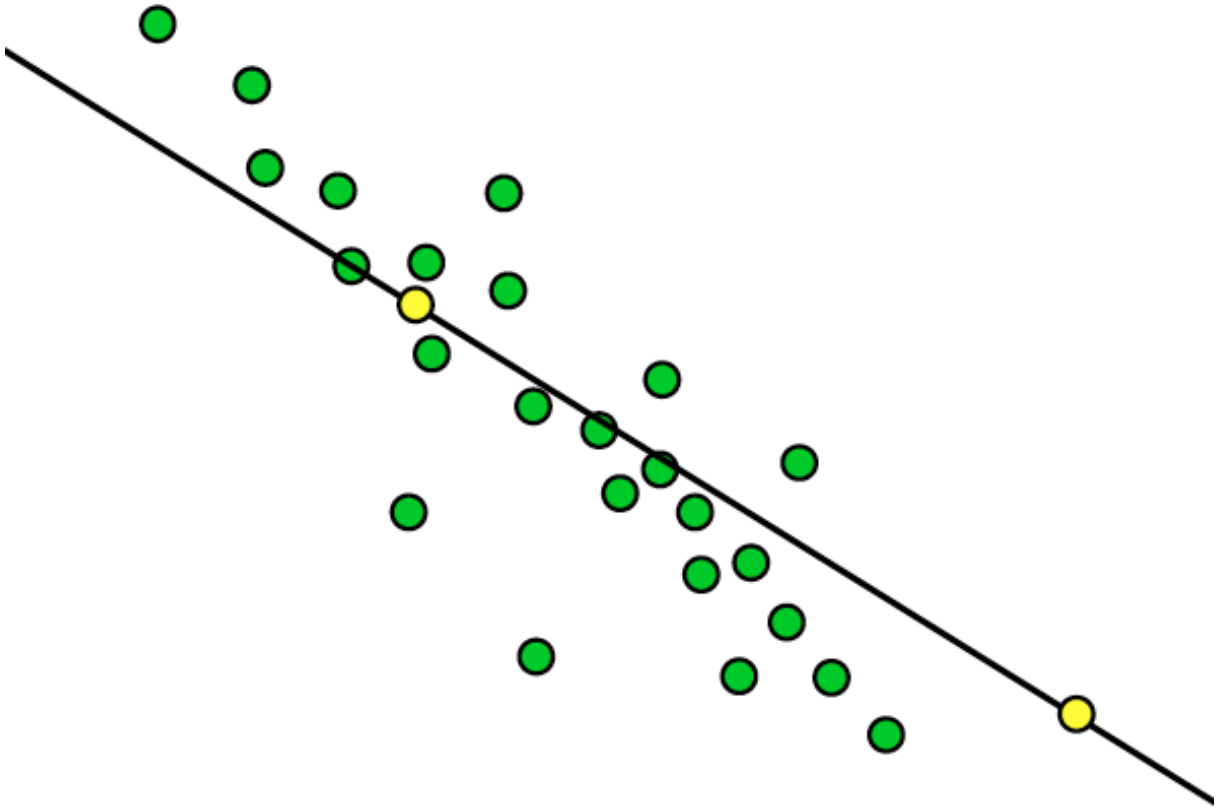


RAndom Sample Consensus (RANSAC)

- Can be used with different models
- Computationally efficient
- RANSAC is non deterministic.
- It solves the fitting problem many times using randomly selected minimal subsets of the data and choosing the best fit.

Count = 19

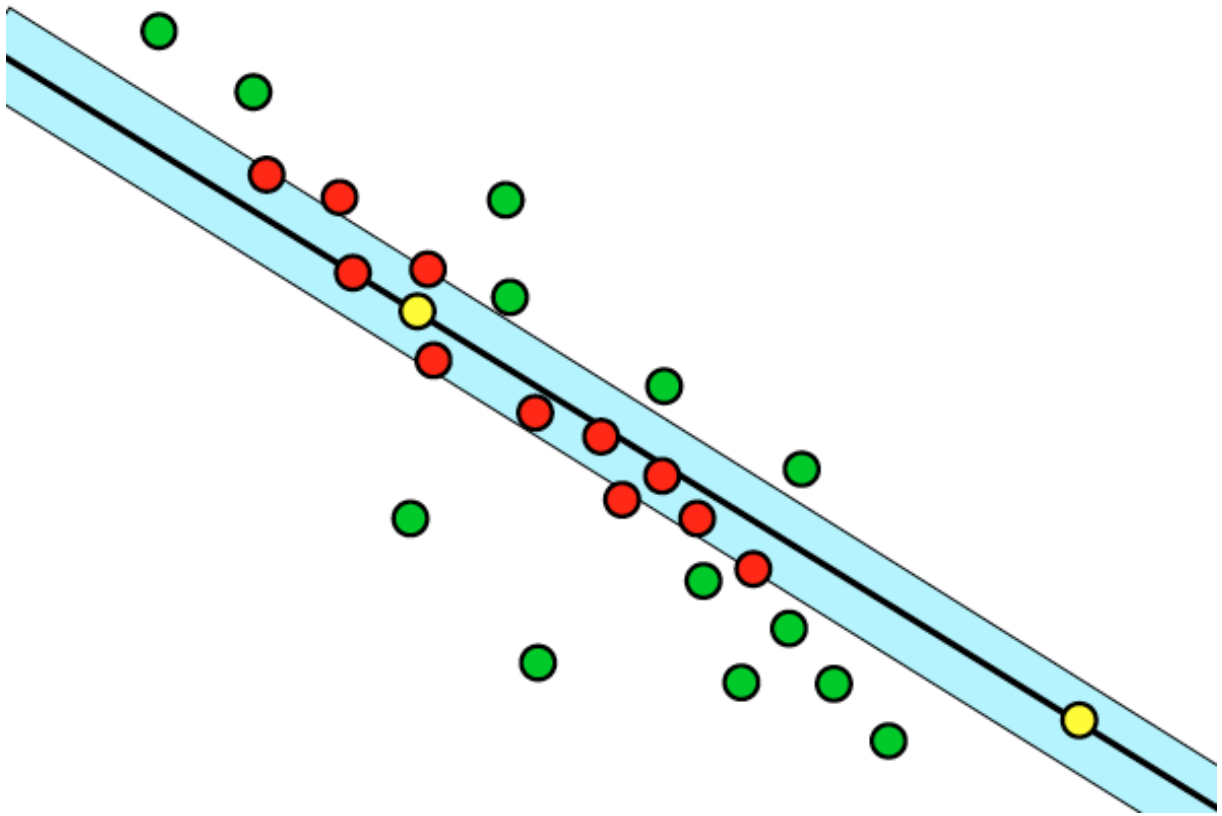
RANSAC



RAndom Sample Consensus (RANSAC)

- Can be used with different models
- Computationally efficient
- RANSAC is non deterministic.
- It solves the fitting problem many times using randomly selected minimal subsets of the data and choosing the best fit.

RANSAC

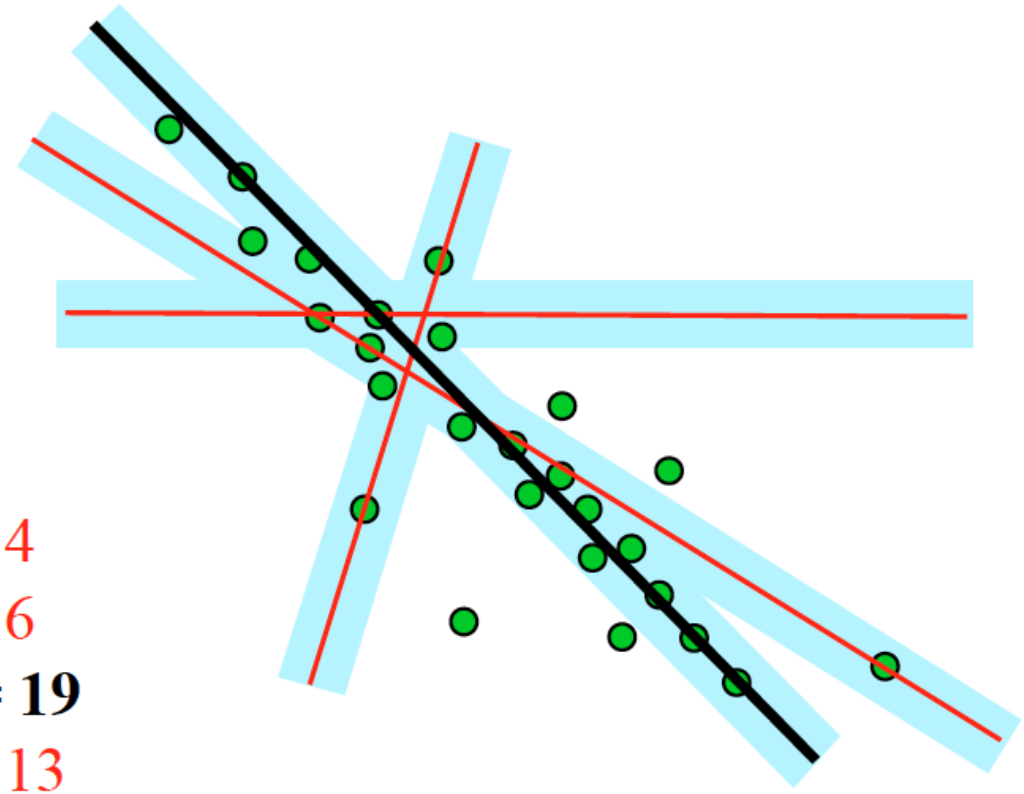


RAndom Sample Consensus (RANSAC)

- Can be used with different models
- Computationally efficient
- RANSAC is non deterministic.
- It solves the fitting problem many times using randomly selected minimal subsets of the data and choosing the best fit.

Count = 13

RANSAC



Count = 4
Count = 6
Count = 19
Count = 13

RAndom Sample Consensus (RANSAC)

- Can be used with different models
- Computationally efficient
- RANSAC is non deterministic.
- It solves the fitting problem many times using randomly selected minimal subsets of the data and choosing the best fit.

RANSAC - Interations

How many iterations does RANSAC need?

Ideally: check all possible combinations of 2 points in a dataset of N points ➔ $\frac{N(N-1)}{2}$.

computationally unfeasible if N is too large.
Example, for 1000 points you need to check all 1000 × 999/2 = 500'000 possibilities!

Do we really need to check all possibilities, or can we stop RANSAC after some iterations?

- it is enough to check a subset of all combinations if we have a rough estimate of the percentage of inliers in our dataset

- This can be done in a probabilistic way

RANSAC - Interations

$\boldsymbol{N}$:= total number of data points

$\boldsymbol{w}$:= number of inliers /N → $\boldsymbol{w}$: fraction of inliers in the dataset → $\boldsymbol{w}$ = P (selecting an inlier point out of the dataset)

Assumption: the 2 points necessary to estimate a line are selected independently

→ $\boldsymbol{w}^2$ = P (both selected points are inliers)

→$\mathbf{1}$-$\boldsymbol{w}^2$ = P (at least one of these two points is an outlier)

Let $\boldsymbol{k}$ be the number of RANSAC iterations executed so far

→ $(\mathbf{1}$ -$\boldsymbol{w}^2)^k$ = P (RANSAC never selected two points that are both inliers after $\boldsymbol{k}$ iterations)

Let $\boldsymbol{p}$:= Probability to have selected at least two points that are both inliers after $\boldsymbol{k}$ iterations. (**Probability of Success)**

→ $\mathbf{1}$ - $\boldsymbol{p}$ = $(\mathbf{1}$ -$\boldsymbol{w}^2)^k$ ⇒ $k = \frac{\log(1-p)}{\log(1-w^2)}$

RANSAC

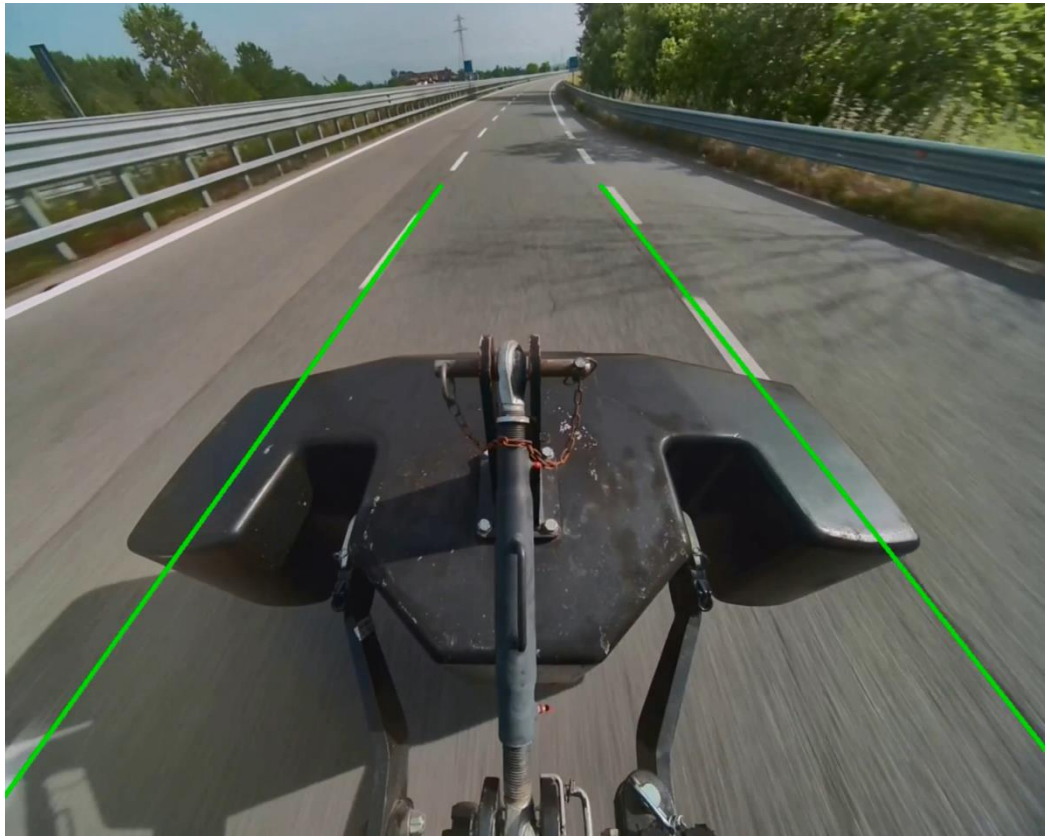
$$k = \frac{\log(1-p)}{\log(1-w^2)}$$

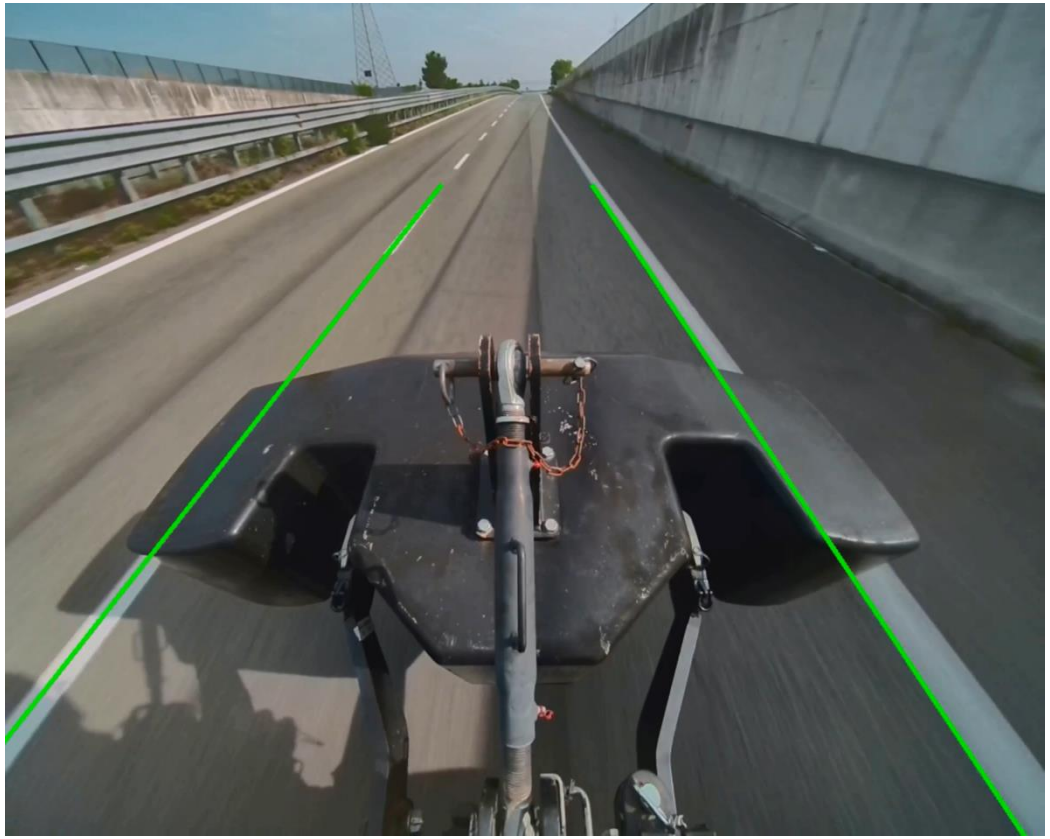
knowing the fraction of inliers $\boldsymbol{w}$, after $\boldsymbol{k}$ iterations we will have a probability $\boldsymbol{p}$ of finding a set of points free of outliers

Example: if we want a probability of success $\boldsymbol{p}$=$\boldsymbol{99}$% and we know that $\boldsymbol{w}$=$\boldsymbol{50}$%→$\boldsymbol{k}$=16 iterations
these are significantly fewer than the number of all possible combinations (500,000)

Notice: the number of data points does not influence the minimum number of iterations $\boldsymbol{k}$, only $\boldsymbol{w}$ does!

RANSAC





Outline

Visual Perception for Autonomous Vehicles

1. Introduction to Cameras
2. Camera Projective Geometry
3. Model Based Image Processing
4. **Neural Network Image Processing**
5. Conclusions

Neural Network Image Processing

In Model Based Image Processing, we leverage apriori knowledge on the scenario.

- We know what to expect from the algorithm. It is developed under well understood assumptions
- In some cases, it is possible to guarantee that it works
- In many cases, the apriori knowledge is difficult to model

Supervised learning –Neural Networks



Neural Network Image Processing

- **Object Detection and Classification:** Identifies and locates specific entities within the frame, typically isolating them with bounding boxes.

  Examples: Recognizing other vehicles, pedestrians, cyclists, and reading traffic signs or traffic light states.

- **Semantic Segmentation:** Classifies every single pixel in an image into a predefined category, providing a complete structural map of the scene.

  Examples: Classifying the "drivable area" (differentiating the road surface from sidewalks or curbs) and extracting lane markings.

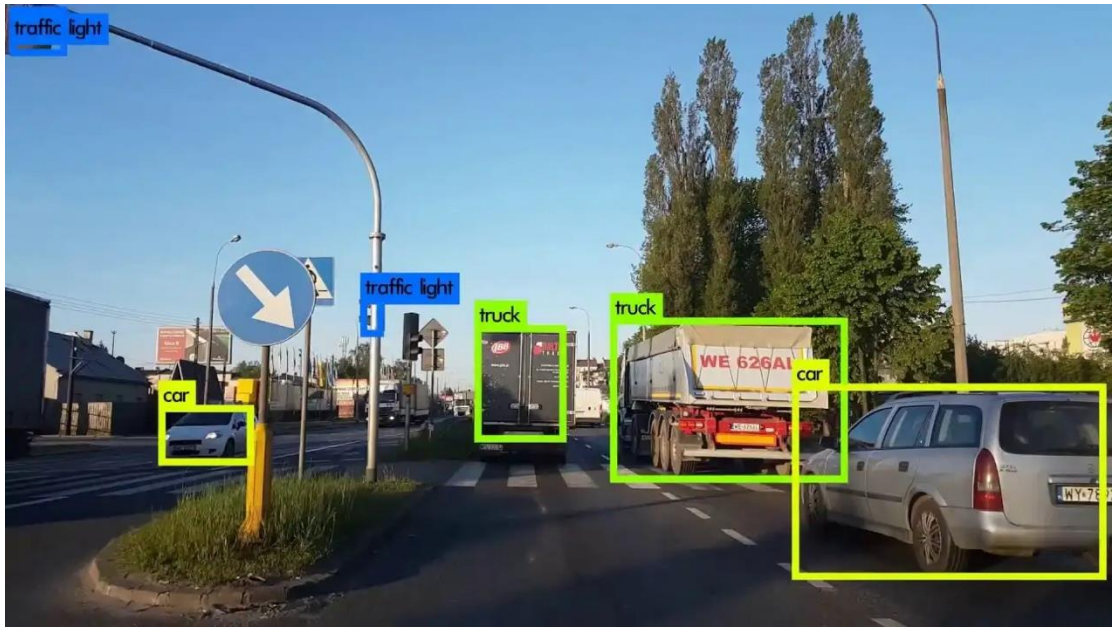
- **Distance Estimation:** Estimate the position of other objects from images

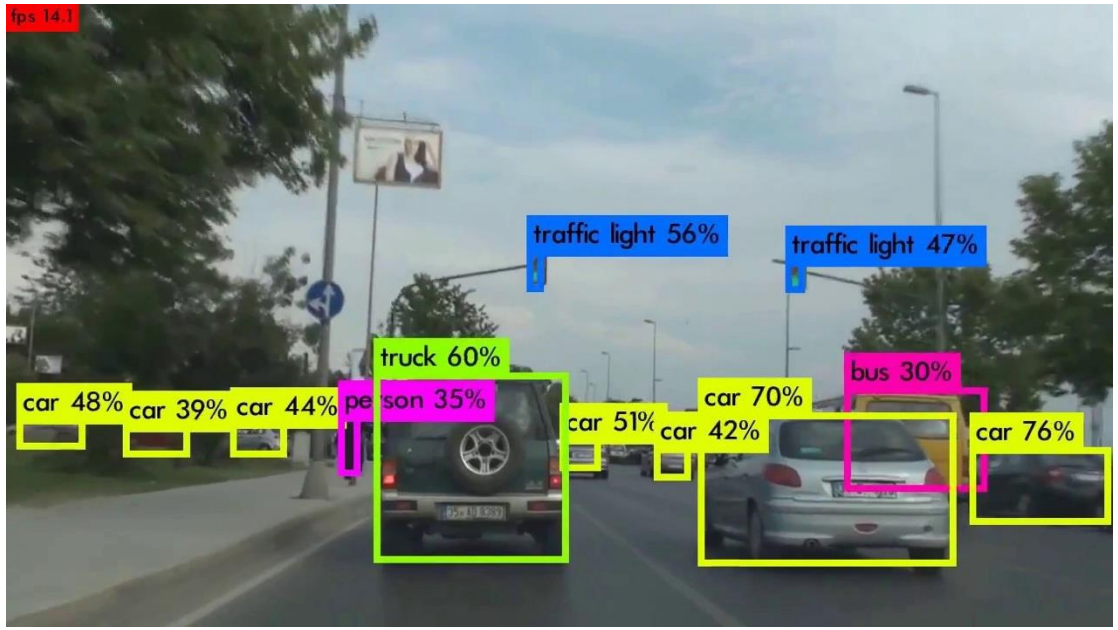
Neural Network Image Processing – Object Detection

- **Object Detection and Classification:** Identifies and locates specific entities within the frame, typically isolating them with bounding boxes.

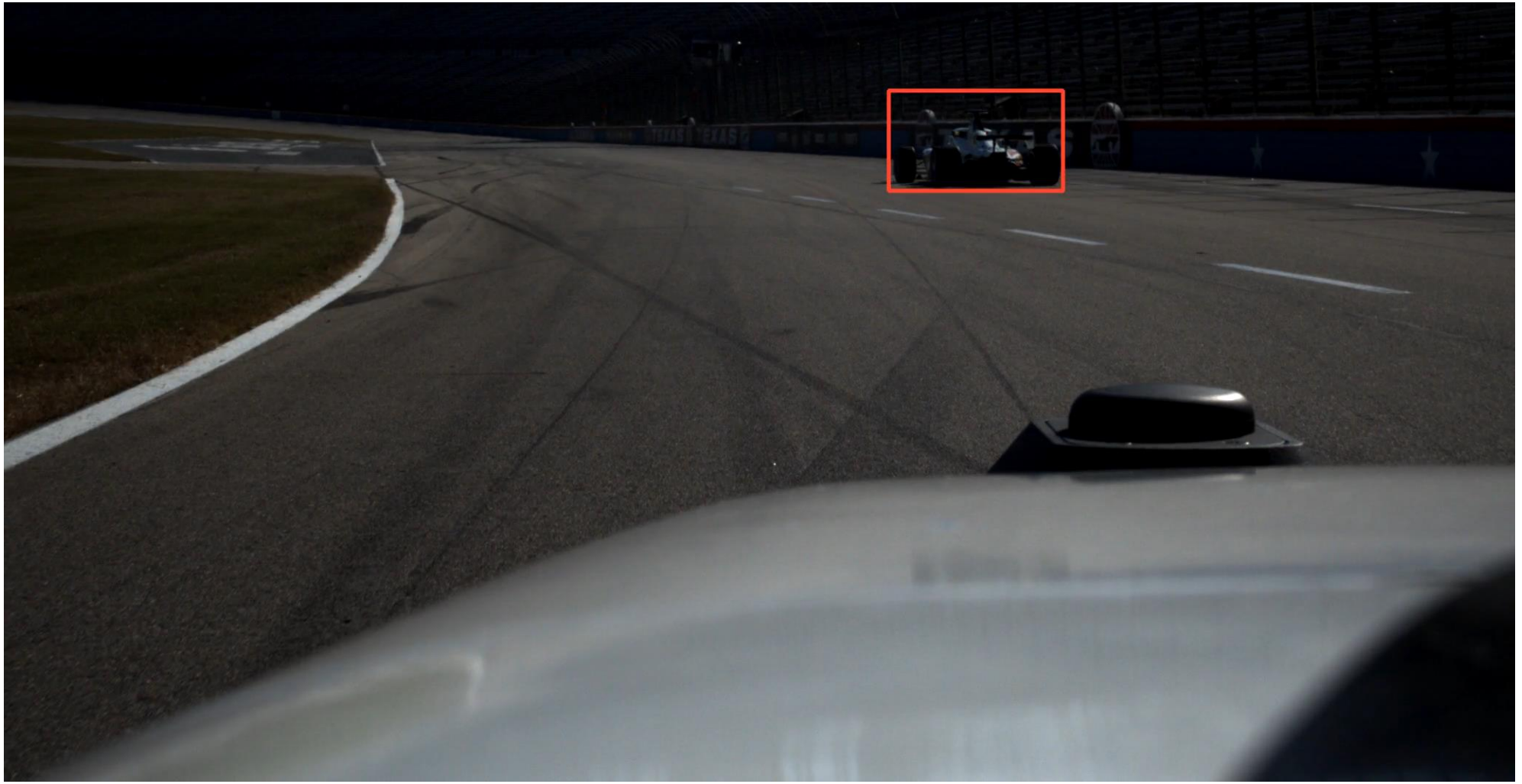
  Examples: Recognizing other vehicles, pedestrians, cyclists, and reading traffic signs or traffic light states.

Usually done on single image





Neural Network Image Processing – Object Detection



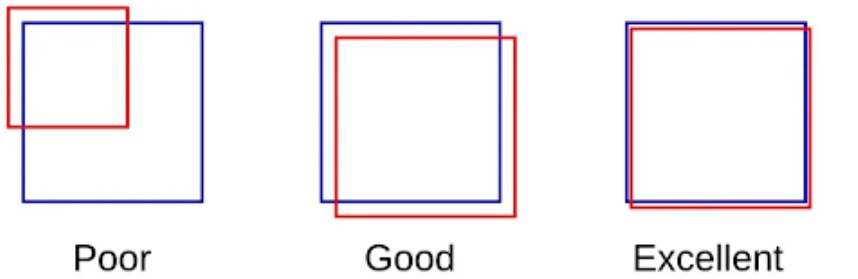
Neural Network Image Processing – Object Detection

- **Object Detection:** Identifies and locates specific entities within the frame, typically isolating them with bounding boxes.

  Examples: Recognizing other vehicles, pedestrians, cyclists, and reading traffic signs or traffic light states.

How to evaluate the results? **Intersection over Union**

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$



Neural Network Image Processing – Object Detection

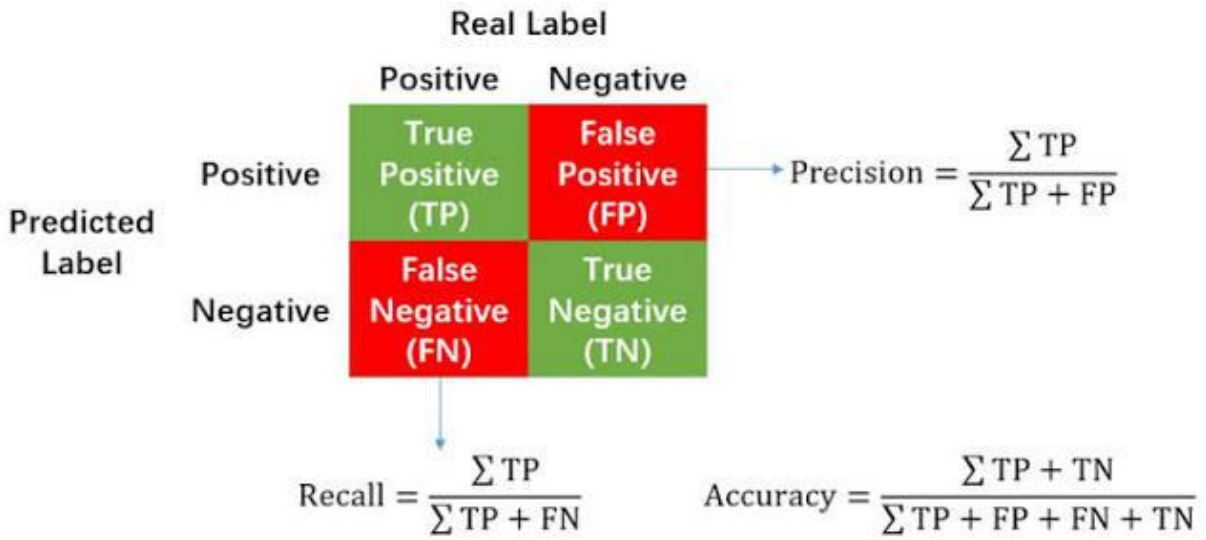
- **Object Detection:** Identifies and locates specific entities within the frame, typically isolating them with bounding boxes.

  Examples: Recognizing other vehicles, pedestrians, cyclists, and reading traffic signs or traffic light states.

How to evaluate the results? **Precision and Recall**

Precision: It quantifies the accuracy of a model's positive predictions (correct). Specifically, it answers the question, **Of all the predictions made by the model, what fraction was correct?**

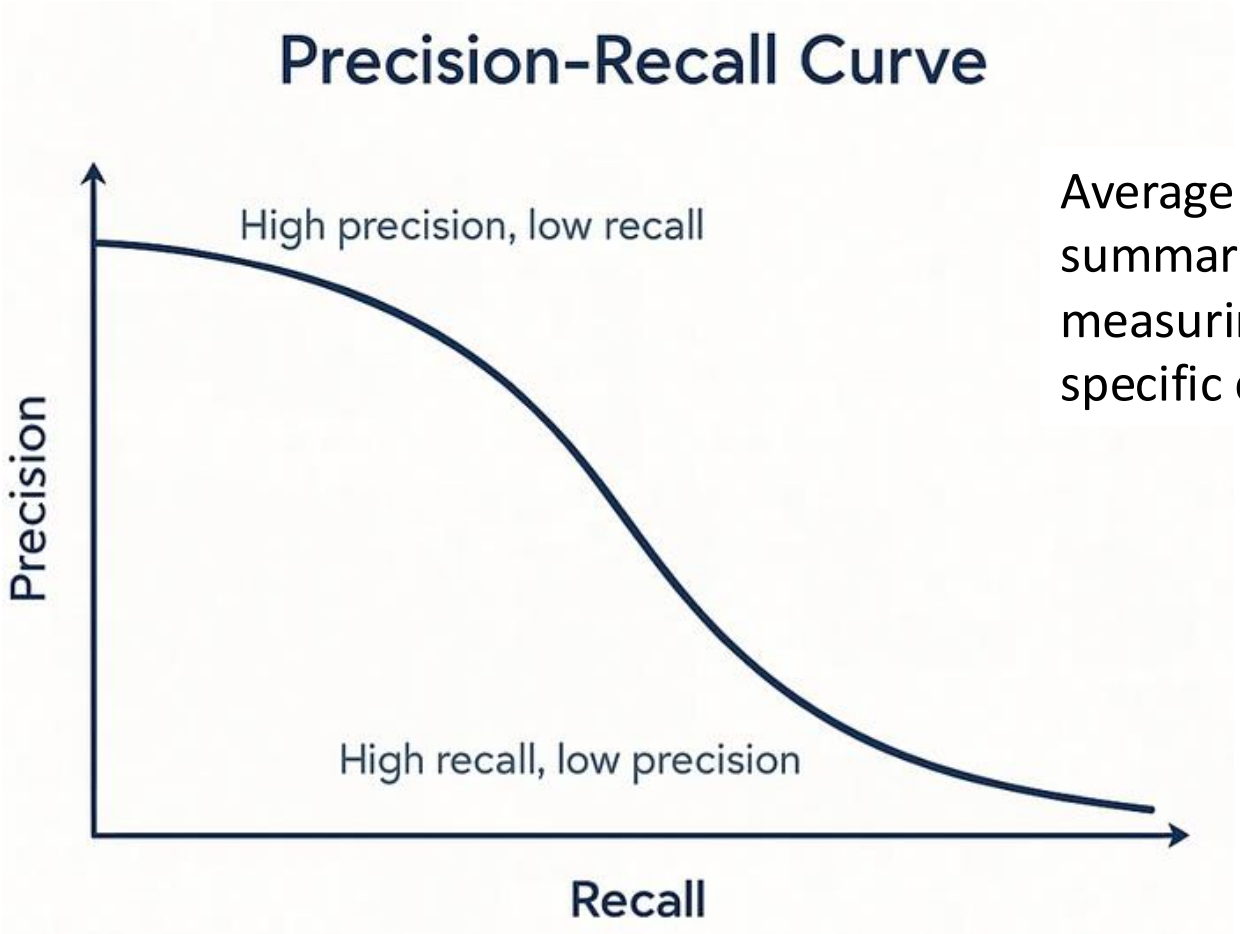
Recall: It measures the ability of a model to find all relevant objects within an image and address the question, **Of all the actual objects in the picture, what fraction did the model find?**



Neural Network Image Processing – Object Detection

- **Object Detection:** Identifies and locates specific entities within the frame, typically isolating them with bounding boxes.

Examples: Recognizing other vehicles, pedestrians, cyclists, and reading traffic signs or traffic light states.

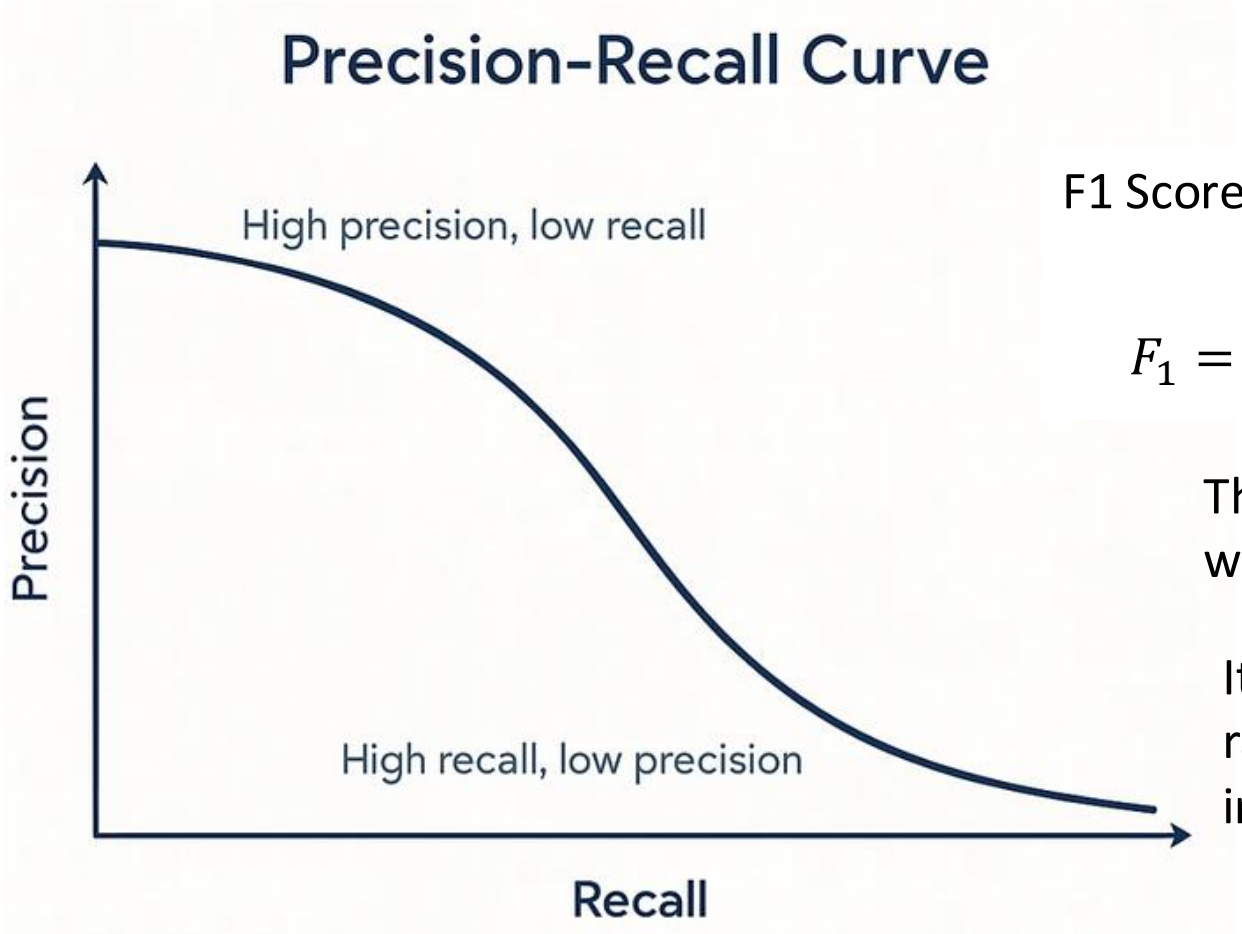


Average Precision (AP) is a single-number metric that summarizes the area under a precision-recall curve, measuring the accuracy and ranking quality of a model for a specific class

Neural Network Image Processing – Object Detection

- **Object Detection:** Identifies and locates specific entities within the frame, typically isolating them with bounding boxes.

  Examples: Recognizing other vehicles, pedestrians, cyclists, and reading traffic signs or traffic light states.



F1 Score

$$F_1 = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1 score is their **harmonic mean**, so it is high only when both are high.

It is especially useful when positive cases are relatively rare, such as pedestrians, cyclists, or hazardous objects in driving images.

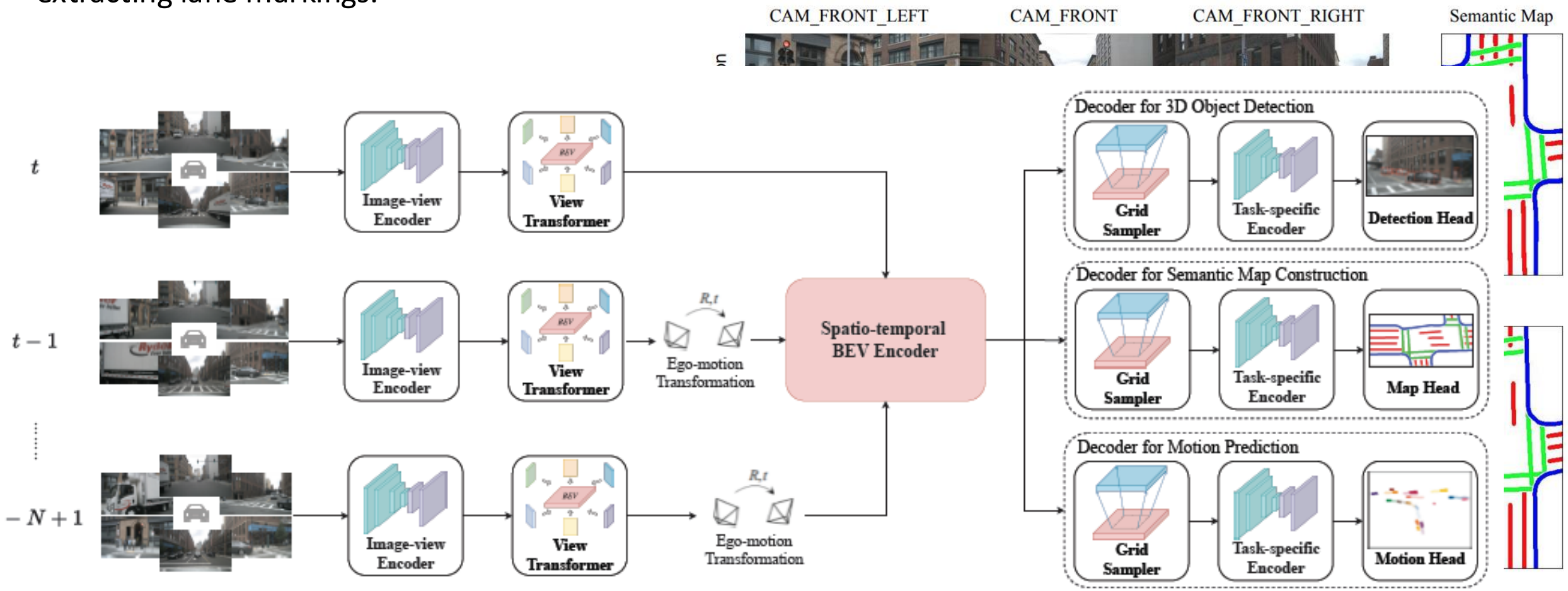
Neural Network Image Processing – Object Detection

	Date	Name	Modalities	Map data	External data	mAP
			Camera	no	no	
>	2025-04-09	RoPETR	Camera	no	no	0.648
>	2024-05-09	HENet_Sp	Camera	no	no	0.645
>	2023-08-01	Far3D	Camera	no	no	0.635
>	2024-03-13	RayDN	Camera	no	no	0.631
>	2024-02-01	HaomoAI Perceptic	Camera	no	no	0.624
>	2023-04-05	HoP	Camera	no	no	0.624
>	2025-06-05	DualViewDistill	Camera	no	no	0.621
>	2023-05-03	StreamPETR-Large	Camera	no	no	0.620
>	2023-06-06	VCD-A	Camera	no	no	0.609
>	2023-08-17	SparseBEV	Camera	no	no	0.603
>	2024-03-11	SparseAD	Camera	no	no	0.592
>	2023-02-07	VideoBEV	Camera	no	no	0.592
>	2022-12-21	BEVDet-Gamma	Camera	no	no	0.586
>	2024-05-09	HENet_BEV	Camera	no	no	0.575

Neural Network Image Processing - Segmentation

- **Semantic Segmentation:** Classifies every single pixel in an image into a predefined category, providing a complete structural map of the scene.

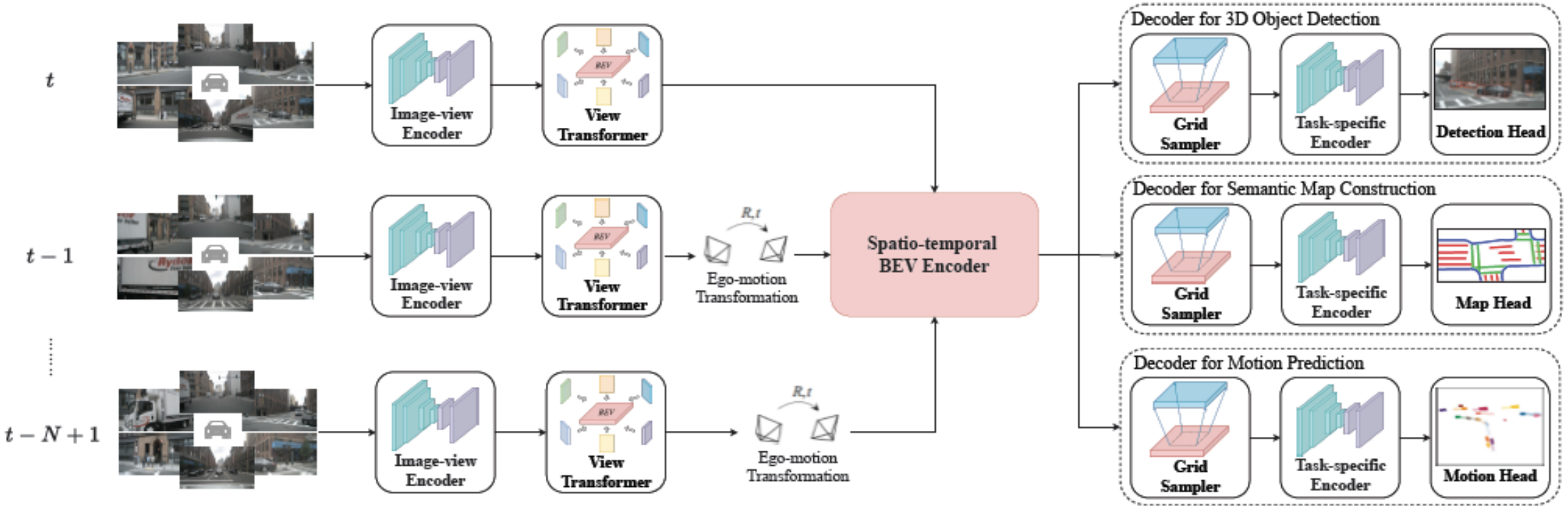
Examples: Classifying the "drivable area" (differentiating the road surface from sidewalks or curbs) and extracting lane markings.



Neural Network Image Processing - Segmentation

- **Semantic Segmentation:** Classifies every single pixel in an image into a predefined category, providing a complete structural map of the scene.

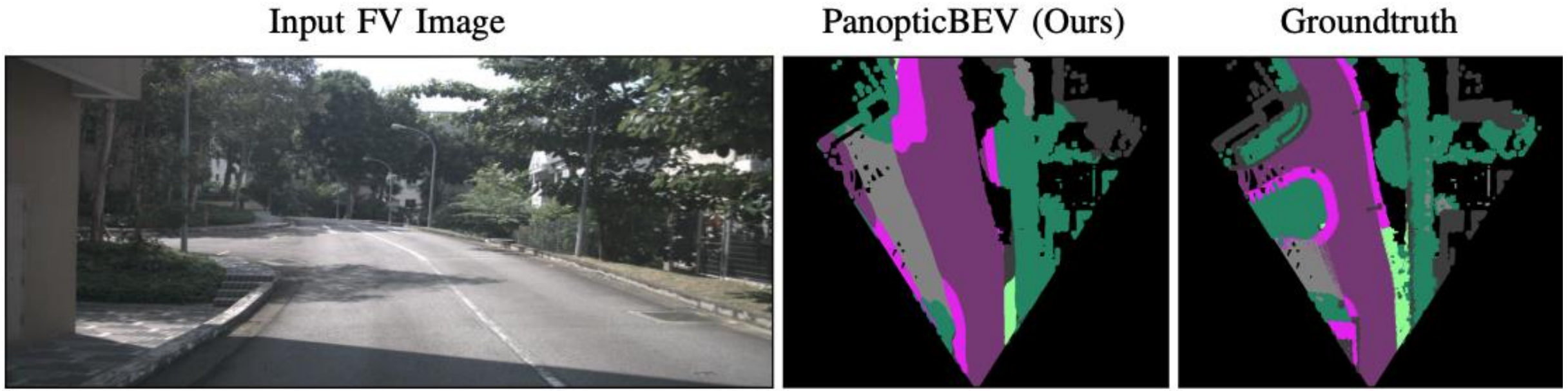
  Examples: Classifying the "drivable area" (differentiating the road surface from sidewalks or curbs) and extracting lane markings.



Neural Network Image Processing - Segmentation

- **Semantic Segmentation:** Classifies every single pixel in an image into a predefined category, providing a complete structural map of the scene.

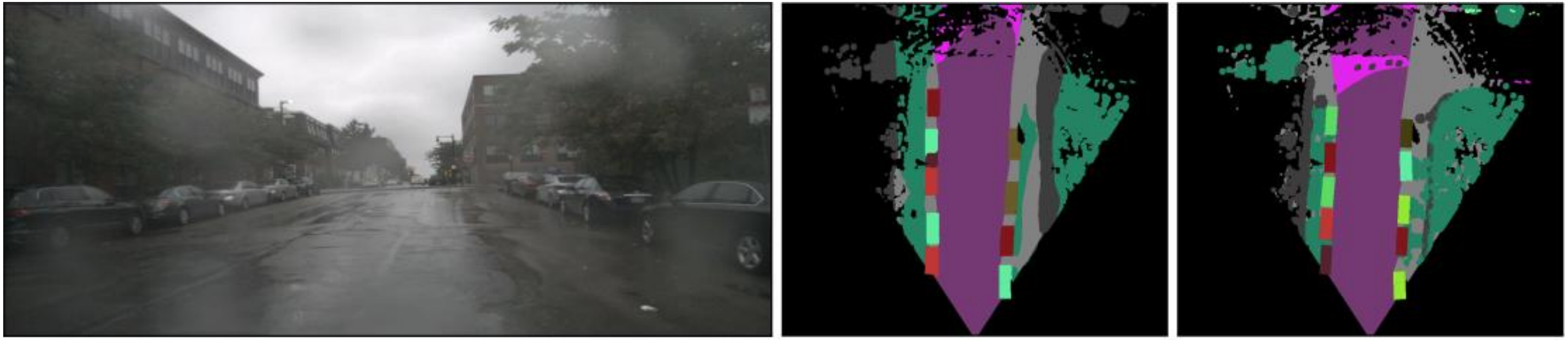
  Examples: Classifying the "drivable area" (differentiating the road surface from sidewalks or curbs) and extracting lane markings.



Neural Network Image Processing - Segmentation

- **Semantic Segmentation:** Classifies every single pixel in an image into a predefined category, providing a complete structural map of the scene.

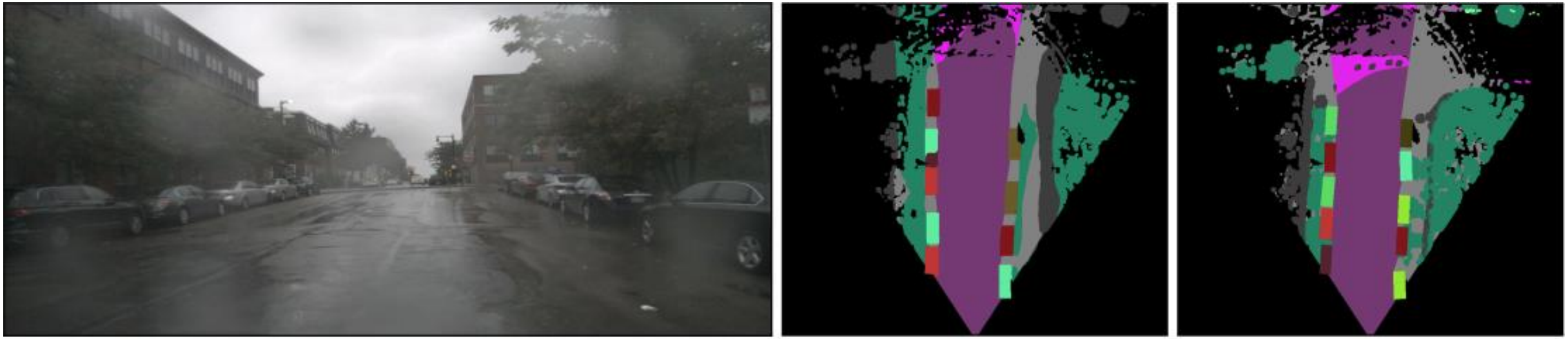
  Examples: Classifying the "drivable area" (differentiating the road surface from sidewalks or curbs) and extracting lane markings.



Neural Network Image Processing - Distance Estimation

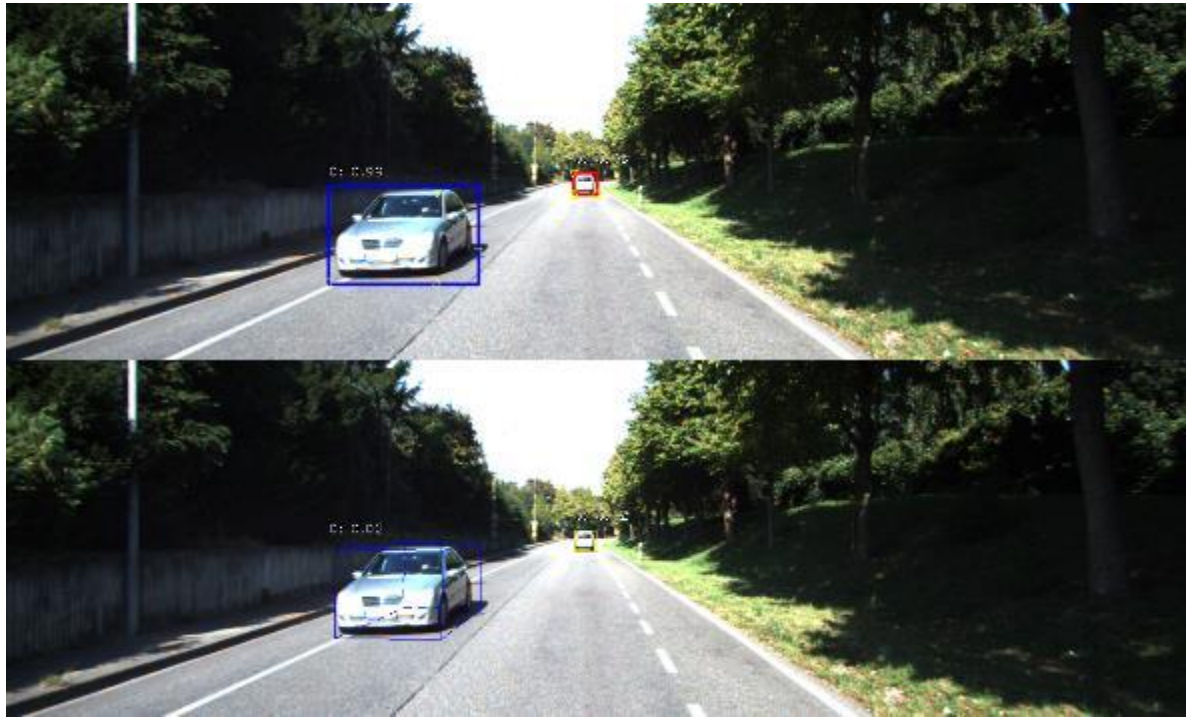
- **Distance Estimation:** Estimate the position of other objects from images

  if the segmentation task is done in the Bird-Eye-View feature space the distance information is embedded in the BEV.



Neural Network Image Processing - Segmentation

- **Position Estimation:** Estimate the position of other objects from images





estimate	X1 [pix]	Y1 [pix]	X2 [pix]	Y2 [pix]	X [m]	Y [m]	Z [m]	Yaw [rad]	Alpha [rad]	
GT	340	195	497	294	-3,31	1,87	13,18	1,57	1,81	
GPP	340	192	497	296	-3,17	1,82	12,86	1,58	1,82	
D3DB	357	198	490	283	-4,07	1,40	16,20	1,70	1,95	
error	X1 [pix]	Y1 [pix]	X2 [pix]	Y2 [pix]	X [m]	Y [m]	Z [m]	Yaw [rad]	Alpha [rad]	d [m]
GPP	0	3	0	2	0,14	0,05	0,32	0,01	0,01	0,35
D3DB	17	3	7	11	0,76	0,47	3,02	0,13	0,14	3,11

estimate	X1 [pix]	Y1 [pix]	X2 [pix]	Y2 [pix]	X [m]	Y [m]	Z [m]	Yaw [rad]	Alpha [rad]	
GT	598	179	620	202	-0,13	2,46	63,27	-1,58	-1,58	
GPP	598	179	622	202	-0,03	2,11	54,14	-1,59	-1,59	
D3DB	593	179	614	200	-0,54	1,35	59,74	-1,51	-1,50	
error	X1 [pix]	Y1 [pix]	X2 [pix]	Y2 [pix]	X [m]	Y [m]	Z [m]	Yaw [rad]	Alpha [rad]	d [m]
GPP	0	0	2	0	0,10	0,35	9,13	0,01	0,01	9,13
D3DB	5	0	6	2	0,41	1,11	3,53	0,07	0,08	3,55

Outline

Visual Perception for Autonomous Vehicles

1. Introduction to Cameras
2. Camera Projective Geometry
3. Model Based Image Processing
4. Neural Network Image Processing
5. **Conclusions**

Conclusions

- Cameras are very powerful tool
- Necessary for some tasks (road sign detection)
- They require a lot of processing
- Processing can be done with
  - algorithmic approaches
  - ML/AI approaches